

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN
BÀI TẬP LỚN MÔN HỆ THỐNG NHÚNG

Hệ thống IoT quan trắc cảm biến

Node WiFi 32-ESP32

Giảng viên:	Trần Nguyễn Minh Duy	
Nhóm:	"L01_02_03 Nhóm 2"	
Sinh viên thực hiện:	Nguyễn Thịnh Thành	2213140
	Cao Thanh Tiến	2213446
	Hồ Đức Tín	2213486
	Hà Chí Trung	2213682
	Nguyễn Văn Tiến	2213467

Mục lục

Danh sách thành viên & đóng góp	4
1 Giới thiệu	5
1.1 Lý do chọn đề tài	5
1.2 Bối cảnh ứng dụng thực tế	5
1.3 Phạm vi của hệ thống	5
1.3.1 Phạm vi về thiết bị phần cứng	5
1.3.2 Phạm vi về nền tảng website và dữ liệu	6
2 Phân tích yêu cầu hệ thống	7
2.1 Yêu cầu chức năng	7
2.2 Yêu cầu phi chức năng	7
3 Thiết kế hệ thống	8
3.1 Sơ đồ khối tổng thể	8
3.1.1 Khối cảm biến và thiết bị đầu cuối (Sensor & End Device)	8
3.1.2 Khối xử lý trung tâm – Embedded IoT Gateway (ESP32)	8
3.1.3 Khối nền đám mây tầng IoT (CoreIoT)	9
3.2 Sơ đồ kết nối phần cứng	10
3.3 Thiết kế phần mềm	10
3.3.1 Luồng thực thi hệ thống	10
3.3.2 Giao thức truyền dữ liệu	11
3.3.2.a Giao tiếp I ² C	11
3.3.2.b Giao tiếp UART	12
3.3.2.c Giao thức RS485	12
3.3.2.d Giao thức MQTT	13
3.3.2.e Giao thức HTTP	13
3.4 Thiết kế giao diện	14
3.4.1 Giao diện Webserver trên ESP32	14
3.4.2 Giao diện Website	17
4 Triển khai	19
4.1 Phần cứng sử dụng	19
4.2 Công cụ phát triển	23
4.3 Các đoạn mã chính	25
4.3.1 Global	25
4.3.2 Main	25
4.3.3 Task Main Server	26
4.3.4 Task CoreIoT	27
4.3.5 Task DHT20	27
4.3.6 Task Air Sensor	28
4.3.7 Task Relay	28
4.3.8 Các task khác	29
4.4 Kết nối tới server	29
5 Kết quả và đánh giá	31
5.1 Kết quả	31
5.2 Đánh giá hiệu năng thực tế	35
5.3 Đối chiếu với mục tiêu ban đầu	35
6 Kết luận và hướng phát triển	36
6.1 Tổng kết các kết quả đạt được	36
6.2 Các giới hạn của hệ thống	36
6.3 Đề xuất hướng cải tiến	36
7 Mã Nguồn	37



Tài liệu tham khảo

37

Danh sách hình vẽ

1	Air quality monitoring system	5
2	Sơ đồ khối hệ thống	8
3	Sơ đồ kết nối phần cứng	10
4	Luồng thực thi hệ thống	10
5	Màn hình Đăng nhập (Login Screen)	17
6	Dashboard nhiệt độ và độ ẩm trên Website	17
7	Dashboard chất lượng không khí trên Website	18
8	Trang Cấu hình và Điều khiển Thiết bị	18
9	Trang Lịch sử Cảnh báo	18
10	Board Yolo UNO	19
11	Cảm biến DHT20	20
12	Cảm biến nồng độ NO ₂ trong không khí	21
13	Cảm biến bụi PM2.5 và PM10	22
14	Visual Studio Code	23
15	PlatformIO	23
16	Mô hình hoạt động của Arduino framework cho ESP32	24
17	FreeRTOS	24
18	Core IoT	29
19	ThingsBoard	29
20	Hệ thống khi lắp đặt thực tế	31
21	ESP32 hoạt động ở chế độ AP và phát ra WiFi	31
22	Dashboard của webserver và dữ liệu cảm biến	32
23	Cấu hình kết nối WiFi và máy chủ MQTT	32
24	Log khi ESP32 nhận được cấu hình từ người dùng	33
25	Log khi ESP32 kết nối thành công với WiFi và máy chủ MQTT	33
26	Dữ liệu cảm biến được gửi đến và lưu trữ tại server Core IoT	33
27	Dữ liệu thu thập được từ cảm biến trong 5 tiếng (11:30 - 16:30)	34
28	Cập nhật firmware từ xa bằng OTA Update của Core IoT	34
29	Log phát hiện bất thường của Tiny ML	34

Danh sách bảng

1	Member list & workload	4
---	------------------------	---



Danh sách thành viên & đóng góp

No.	Fullname	Student ID	% Done
1	Nguyễn Thịnh Thành	2213140	100%
2	Cao Thanh Tiến	2213446	100%
3	Hồ Đức Tín	2213486	100%
4	Hà Chí Trung	2213682	100%
5	Nguyễn Văn Tiến	2213467	100%

Bảng 1: *Member list & workload*

1 Giới thiệu

1.1 Lý do chọn đề tài

Nhu cầu giám sát môi trường, đặc biệt là chất lượng không khí, đang trở nên cấp thiết (urgent) do quá trình đô thị hóa nhanh chóng và sự gia tăng của các nguồn ô nhiễm. Chất lượng không khí ảnh hưởng trực tiếp đến sức khỏe con người và hiệu suất làm việc trong các môi trường kín như phòng thí nghiệm, văn phòng hay nhà ở. Việc tích hợp cảm biến nhiệt độ và độ ẩm là cần thiết để đánh giá toàn diện môi trường, vì các yếu tố này ảnh hưởng đến sự thoải mái và độ chính xác của các cảm biến hóa học khác.



Hình 1: *Air quality monitoring system*

Mục tiêu của dự án là xây dựng một **Hệ thống IoT quan trắc chất lượng không khí** toàn diện, cung cấp khả năng thu thập dữ liệu chính xác, truyền tải dữ liệu theo thời gian thực, và trực quan hóa thông qua một nền tảng website do nhóm phát triển. Hệ thống này cho phép người dùng theo dõi các thông số quan trọng và đưa ra các hành động kịp thời khi chất lượng không khí vượt ngưỡng an toàn.

1.2 Bối cảnh ứng dụng thực tế

Hệ thống được thiết kế để áp dụng trong các môi trường yêu cầu kiểm soát chất lượng môi trường nghiêm ngặt, chẳng hạn như:

- **Phòng thí nghiệm/Phòng sạch:** Giám sát nồng độ CO_2 , bụi mịn ($PM_{2.5}$, PM_{10}), và các thông số khí hậu để đảm bảo an toàn và tính toàn vẹn của thí nghiệm.
- **Không gian sống và làm việc:** Cung cấp dữ liệu về nhiệt độ, độ ẩm, và chất lượng không khí, hỗ trợ điều chỉnh hệ thống thông gió hoặc lọc không khí tự động.

1.3 Phạm vi của hệ thống

1.3.1 Phạm vi về thiết bị phần cứng

Phạm vi của hệ thống giới hạn ở việc sử dụng các thiết bị và cảm biến dành cho mục đích nghiên cứu, học tập và ứng dụng dân dụng cơ bản:

- **Vi điều khiển:** Hệ thống sử dụng dòng vi điều khiển ESP32-S3 - là thiết bị nhúng công suất thấp, được thiết kế với kích thước nhỏ gọn và tài nguyên bộ nhớ phù hợp cho các bài toán IoT/AI nhẹ.

- **Thiết bị điều khiển và cảnh báo:** Hệ thống sử dụng các thiết bị đơn giản, công suất thấp để mô phỏng cảnh báo như đèn LED nhỏ cũng như các thiết bị quạt mini. Hệ thống trong dự án này không bao gồm các thiết bị công suất cao như trong thực tế (Quạt, LED điện áp 220V).

1.3.2 Phạm vi về nền tảng website và dữ liệu

Phạm vi của nền tảng Backend và cơ sở dữ liệu của website tự phát triển được xác định như sau:

- **Nền tảng IoT và Sever:** Hệ thống không tự xây dựng máy chủ (server) và cơ sở dữ liệu vật lý. Thay vào đó, hệ thống sử dụng Core IoT, một server IoT làm nền tảng trung gian và máy chủ dữ liệu chính.
- **Chức năng của server bên thứ ba:** Core IoT chịu trách nhiệm xử lý các tác vụ:
 - **Thu thập và lưu trữ dữ liệu:** Tiếp nhận và lưu trữ dữ liệu gửi lên từ vi điều khiển.
 - **Cung cấp dữ liệu cho website:** Dữ liệu được Core IoT lưu trữ sẽ được website do nhóm tự phát triển kết nối vào và truy xuất dữ liệu.
- **Website Dashboard:** Là một website tự phát triển đóng vai trò là giao diện người dùng cho phép người dùng quan sát và thay đổi một số thông số của vi điều khiển. Web site này không kết nối trực tiếp đến thiết bị mà giao tiếp và truy xuất dữ liệu thông qua Core IoT làm trung gian.

2 Phân tích yêu cầu hệ thống

2.1 Yêu cầu chức năng

Các yêu cầu chức năng định nghĩa các hành vi cụ thể mà hệ thống phải thực hiện để đáp ứng mục tiêu quan trắc và điều khiển.

- **Chức năng thu thập dữ liệu đa thông số:** Hệ thống phải định kỳ (periodically) đọc dữ liệu từ các cảm biến chính:
 - Xác định nồng độ bụi mịn $PM_{2.5}$, PM_{10} .
 - Đo nồng độ Carbon Dioxide (CO_2) trong không khí.
 - Đo nhiệt độ và độ ẩm trong không khí.
- **Chức năng Giao tiếp và Truyền thông:** Thiết bị vi điều khiển cung cấp hai khả năng chính để người dùng tùy chọn:
 - Khi thiết bị chưa kết nối WiFi, vi điều khiển sẽ hoạt động ở chế độ AP mode cho phép người dùng kết nối đến server của chính nó và chọn WiFi hiện có để thiết bị kết nối vào ở chế độ STA mode.
 - Sau khi thiết bị được người dùng thiết lập kết nối wifi, nó sẽ hoạt động ở chế độ STA mode, gửi dữ liệu cảm biến đến IoT server để người dùng có thể quan sát.
- **Chức năng hiển thị và thiết lập cấu hình:** Hệ thống bao gồm một website tự phát triển cho phép người dùng:
 - Quan sát dữ liệu thu thập được từ cảm biến thông qua dashboard của website theo thời gian thực.
 - Cung cấp trang cấu hình cho phép người dùng thay đổi các thông số cũng như điều khiển từ xa vi điều khiển.
- **Xử lý AI tại biên và thiết lập điều kiện kích hoạt cục bộ:** Hệ thống thực hiện các hành động tự động dựa trên ngưỡng được thiết lập:
 - Điều kiện ngưỡng: Khi giá trị cảm biến vượt ngưỡng nguy hiểm, hệ thống tự động kích hoạt cơ chế tự thiết lập cục bộ.
 - Cảnh báo khi phát hiện bất thường bằng mô hình AI nhẹ (tiny AI) được tích hợp ngay trên vi điều khiển.

2.2 Yêu cầu phi chức năng

Các yêu cầu phi chức năng tập trung vào các tiêu chí chất lượng về cách thức hệ thống vận hành.

- **Hiệu suất:** Thời gian trễ trung bình từ thiết bị đến Dashboard không được vượt quá một ngưỡng tối đa đối với truyền tải dữ liệu.
- **Khả năng mở rộng:** Kiến trúc phần mềm phải đảm bảo tính modular để cho phép dễ dàng tích hợp các loại cảm biến hoặc thiết bị điều khiển mới mà không cần thay đổi kiến trúc cốt lõi.
- **Độ tin cậy:** Thiết bị phải có cơ chế tự động kết nối lại sau khi phát hiện mất kết nối mạng.

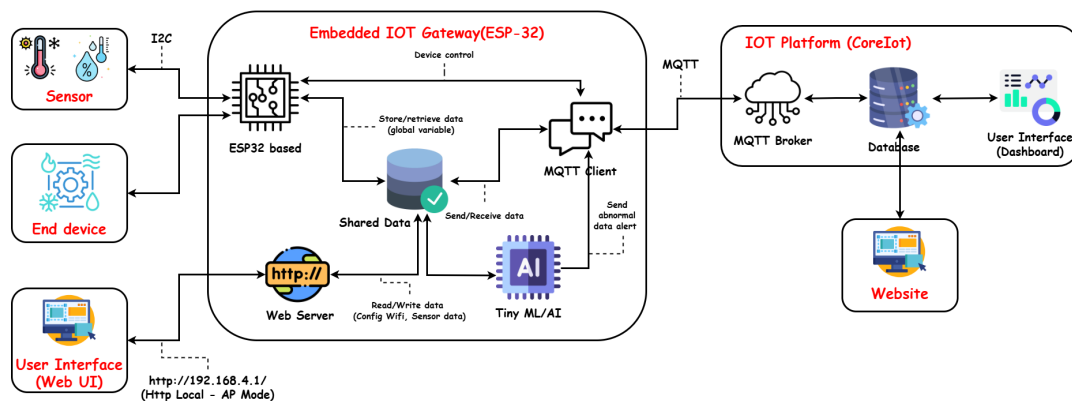
3 Thiết kế hệ thống

3.1 Sơ đồ khối tổng thể

Hệ thống IoT quan trắc môi trường được thiết kế theo mô hình kiến trúc phân tầng, bao gồm ba thành phần chính:

1. Khối cảm biến và thiết bị đầu cuối (Sensor & End Device)
2. Khối xử lý trung tâm – Embedded IoT Gateway (ESP32)
3. Khối nền tảng đám mây IoT (CoreIoT)

Các thành phần này phối hợp với nhau để thu thập dữ liệu, xử lý, điều khiển thiết bị và hiển thị thông tin cho người dùng theo thời gian thực. Sơ đồ khối tổng thể được mô tả như hình dưới đây:



Hình 2: Sơ đồ khối hệ thống

3.1.1 Khối cảm biến và thiết bị đầu cuối (Sensor & End Device)

Khối này bao gồm các cảm biến đo nhiệt độ, độ ẩm hoặc các thiết bị đầu cuối như DHT22, MQ135, BH1750... được kết nối với ESP32 thông qua giao thức I2C, GPIO hoặc ADC tùy loại. Nhiệm vụ của khối này là:

- Thu thập dữ liệu môi trường.
- Gửi dữ liệu thô về bộ xử lý chính - ESP32.
- Nhận lệnh điều khiển từ ESP32 trong trường hợp có thiết bị chấp hành (relay, motor...).

3.1.2 Khối xử lý trung tâm – Embedded IoT Gateway (ESP32)

Khối xử lý trung tâm của hệ thống được xây dựng trên nền tảng vi điều khiển ESP32, đảm nhiệm vai trò như một Embedded IoT Gateway. Đây là thành phần cốt lõi, đóng vai trò như “bộ não” của toàn hệ thống, chịu trách nhiệm thực hiện chuỗi tác vụ chính bao gồm: thu thập dữ liệu từ cảm biến, xử lý và phân tích thông tin, điều khiển thiết bị và truyền tải dữ liệu lên nền tảng IoT.

Toàn bộ quá trình vận hành của ESP32 được xây dựng dựa trên FreeRTOS cho phép quản lý song song nhiều tác vụ. Ngoài ra các chức năng được triển khai thông qua các module phần mềm tích hợp, bao gồm:

a. Đọc dữ liệu cảm biến và điều khiển thiết bị

- ESP32 định kỳ đọc dữ liệu từ cảm biến.
- Lưu dữ liệu tạm thời vào vùng Shared Data.
- Điều khiển thiết bị đầu cuối dựa trên dữ liệu hoặc trên yêu cầu từ phía server/MQTT.

b. WebServer cấu hình

Khi thiết bị lần đầu được cấp nguồn, hoặc không thể kết nối WiFi do cấu hình chưa có hoặc sai, ESP32 sẽ tự chuyển sang chế độ Access Point, hoặc có thể chuyển chế độ thủ công để người dùng cấu hình thiết bị. ESP32 tạo một Access Point (AP) với địa chỉ: `http://192.168.4.1`. Tại giao diện WebServer, người dùng có thể thực hiện các thao tác cấu hình sau:

- Thiết lập thông tin mạng WiFi: SSID và mật khẩu kết nối.
- Cài đặt thông số MQTT: địa chỉ broker, cổng, topic và thông tin xác thực.
- Thêm hoặc cấu hình relay điều khiển thiết bị đầu cuối.
- Thiết lập ngưỡng cảnh báo cho các cảm biến (ví dụ: nhiệt độ, độ ẩm, chất lượng không khí).
- Thực hiện cập nhật firmware OTA: cho phép nâng cấp phần mềm từ xa thông qua giao diện web mà không cần can thiệp phần cứng.

Dữ liệu cấu hình sau đó được lưu lại trong Flash để đảm bảo bền vững sau khi khởi động lại, đảm bảo dữ liệu không bị mất khi thiết bị khởi động lại, mất điện hay cập nhật firmware OTA.

c. TinyML/AI

Hệ thống được tích hợp một mô hình học máy nhẹ (TinyML) trực tiếp trên vi điều khiển ESP32, cho phép thực hiện phân tích dữ liệu ngay tại thiết bị mà không cần phụ thuộc vào nền tảng đám mây. Khi phát hiện bất thường, hệ thống:

- Gửi cảnh báo thời gian thực lên nền tảng CoreIoT qua MQTT.
- Ghi log sự kiện vào bộ nhớ chia sẻ.

d. MQTT Client

ESP32 hoạt động như một client kết nối tới nền tảng CoreIoT thông qua giao thức MQTT. Các chức năng chính bao gồm:

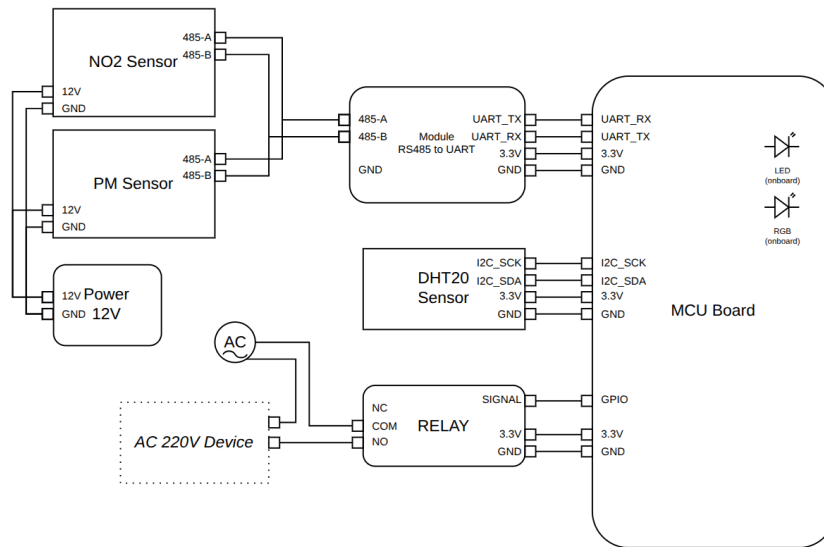
- Gửi dữ liệu cảm biến định kỳ lên CoreIoT để lưu trữ và hiển thị trên dashboard.
- Nhận lệnh điều khiển từ dashboard, thực thi trực tiếp trên các thiết bị đầu cuối (relay, actuator).
- Phát cảnh báo thời gian thực khi mô hình AI/TinyML tại thiết bị phát hiện bất thường, thông qua topic cảnh báo (alert).
- Cách triển khai này giúp hệ thống duy trì luồng dữ liệu hai chiều ổn định, hỗ trợ giám sát và điều khiển từ xa một cách hiệu quả.

3.1.3 Khối nền đám mây tầng IoT (CoreIoT)

Trong hệ thống, CoreIoT đóng vai trò là nền tảng trung tâm, đảm bảo việc thu thập, lưu trữ, phân tích và hiển thị dữ liệu từ các thiết bị ESP32. Đây là lớp hạ tầng đám mây giúp hệ thống đạt được khả năng giám sát và điều khiển từ xa một cách ổn định, an toàn và mở rộng.

Trên nền tảng này, dashboard trực quan được xây dựng nhằm hiển thị dữ liệu môi trường dưới dạng biểu đồ, bảng và cảnh báo, cho phép người dùng dễ dàng theo dõi xu hướng và trạng thái thiết bị. Bên cạnh đó, dự án còn phát triển một *website quản lý riêng*, sử dụng API của CoreIoT để truy xuất dữ liệu từ cơ sở dữ liệu. Website này mở rộng khả năng giám sát và điều khiển từ xa, đồng thời cho phép tùy biến giao diện và chức năng theo nhu cầu thực tế.

3.2 Sơ đồ kết nối phần cứng

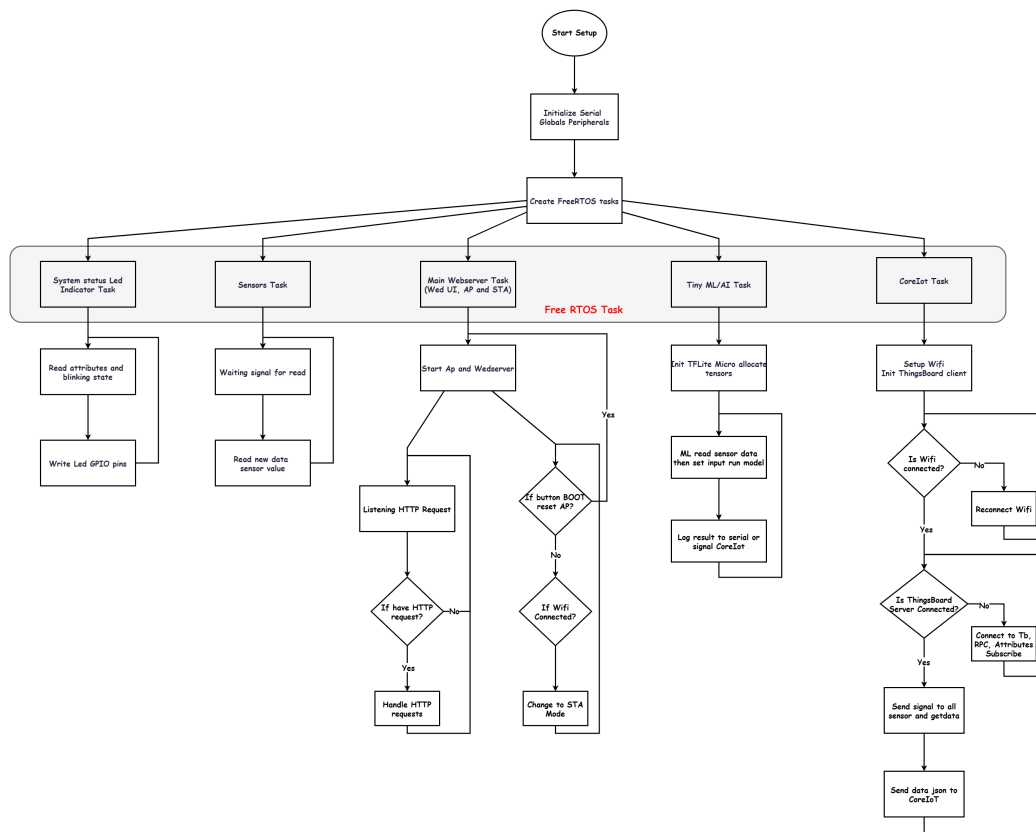


Hình 3: Sơ đồ kết nối phần cứng

3.3 Thiết kế phần mềm

3.3.1 Luồng thực thi hệ thống

Hệ thống được thiết kế theo mô hình tuần tự kết hợp đa nhiệm, ứng dụng FreeRTOS để vận hành song song nhiều nhiệm vụ phục vụ giám sát môi trường và IoT, trong đó ESP32 đóng vai trò trung tâm, thực hiện các bước từ khởi tạo đến giám sát và truyền dữ liệu.



Hình 4: Luồng thực thi hệ thống

Luồng thực thi tổng thể chi tiết như sau:

1. *Khởi động*

- Bootloader ESP32 nạp firmware, chương trình bắt đầu thực thi.
- Khởi tạo phần cứng: UART (serial), I/O, ADC, I2C, LED, relay,...

2. *Khởi tạo phần mềm*

- Khởi tạo hệ thống file LittleFS, có chứa web UI.
- Khởi tạo NVS để đọc cấu hình lưu trữ Wi-Fi, MQTT,....

3. *Kiểm tra cấu hình mạng*

- Nếu chưa có cấu hình Wi-Fi hợp lệ, chuyển sang Access Point Mode.
- Nếu có cấu hình và Wi-Fi khả dụng, cố gắng kết nối Wi-Fi.

4. *Chế độ AP*

- ESP32 bật AP và khởi chạy WebServer (<http://192.168.4.1>).
- Người dùng truy cập trang cấu hình, nhập SSID/Password, MQTT,....
- Khi người dùng nhấn lưu, ghi cấu hình vào NVS/Preferences, sau đó device khởi động lại, chuyển sang client mode và kết nối Wi-Fi.

5. *Kết nối mạng & dịch vụ*

- Kết nối Wi-Fi thành công, tạo kết nối tới CoreIoT
- Nếu có OTA hoặc các service khác khởi tạo tương ứng.

6. *Chu trình thu thập & xử lý*

- Khi có tín hiệu, các cảm biến đọc giá trị, tổng hợp gửi cho coreiot_task thông qua đồng bộ queue.
- tinymtl_task lấy dữ liệu từ Shared Data, chạy inference, nếu phát hiện anomaly, ghi log & push event vào queue cảnh báo.
- coreiot_task lấy dữ liệu từ Shared Data hoặc queue publish lên topic telemetry; subscribe topic command để nhận lệnh điều khiển.

3.3.2 Giao thức truyền dữ liệu

Hệ thống sử dụng ba giao thức truyền thông chính gồm I²C, UART và RS485. Việc lựa chọn các giao thức này dựa trên đặc tính phần cứng của cảm biến, yêu cầu về tốc độ truyền, khả năng chống nhiễu và độ tin cậy dữ liệu trong môi trường thực tế.

3.3.2.a Giao tiếp I²C

Giao thức I²C được sử dụng để kết nối vi điều khiển ESP32 với cảm biến DHT20. Giao thức hoạt động trên hai đường truyền:

- **SDA**: truyền và nhận dữ liệu hai chiều.
- **SCL**: tạo xung nhịp điều khiển quá trình truyền dữ liệu.

Cảm biến DHT20 hoạt động ở địa chỉ cố định 0x38. Tốc độ truyền được thiết lập ở mức 100 kHz nhằm đảm bảo độ ổn định và hạn chế nhiễu. Vi điều khiển định kỳ truy vấn cảm biến và thu thập dữ liệu nhiệt độ – độ ẩm để phục vụ cho quá trình xử lý và giám sát.

3.3.2.b Giao tiếp UART

UART được sử dụng làm tầng truyền thông giữa ESP32 và module chuyển đổi RS485. Giao thức truyền dữ liệu bất đồng bộ qua hai chân:

- **TX**: truyền dữ liệu từ vi điều khiển.
- **RX**: nhận dữ liệu từ module giao tiếp.

UART trong hệ thống được cấu hình theo chuẩn 9600 baud, khung truyền 8N1 (8 bit dữ liệu, không parity, 1 bit stop). Tín hiệu UART sau đó được chuyển đổi sang tín hiệu vi sai RS485 thông qua IC MAX485, giúp tăng khả năng chống nhiễu và hỗ trợ truyền dữ liệu ở khoảng cách lớn.

3.3.2.c Giao thức RS485

RS485 được sử dụng làm tầng truyền dẫn vật lý để giao tiếp với cảm biến NO2 và PM. Ở tầng ứng dụng, giao thức truyền dữ liệu được triển khai theo chuẩn Modbus RTU nhằm đảm bảo tính thống nhất, độ tin cậy và khả năng kiểm tra lỗi thông qua mã CRC16.

Tổng quan thiết kế

- **Tầng vật lý**: sử dụng đường truyền vi sai A/B theo chế độ half-duplex.
- **Tầng giao tiếp**: ESP32 điều khiển module MAX485 để chuyển đổi giữa chế độ truyền và nhận dữ liệu thông qua hai chân DE và RE.
- **Tầng ứng dụng**: sử dụng khung truyền Modbus RTU với mã chức năng, địa chỉ thanh ghi và CRC16 kiểm tra lỗi.

Cấu hình giao tiếp với cảm biến

- **Slave ID**: 35.
- **Địa chỉ các thanh ghi**:
 - NO2 Content (N): 0x0006.

Cấu trúc khung Modbus RTU Gói yêu cầu đọc một thanh ghi (Function code 0x03):

[ID][0x03][AddrHi][AddrLo][0x00][0x01][CRC-Lo][CRC-Hi]

Ví dụ yêu cầu đọc thanh ghi 0x001E:

01 03 00 1E 00 01 CRC-L CRC-H

Gói phản hồi:

[ID][0x03][0x02][DataHi][DataLo][CRC-Lo][CRC-Hi]

Ví dụ dữ liệu phản hồi 0x002D:

01 03 02 00 2D CRC-L CRC-H

Luồng trao đổi dữ liệu

1. Vi điều khiển lần lượt gửi yêu cầu đọc ba thanh ghi NO2, PM2.5, PM10.
2. Cảm biến phản hồi bằng gói dữ liệu Modbus RTU chứa giá trị đo.
3. Gói dữ liệu được kiểm tra tính hợp lệ thông qua ID, mã chức năng và CRC.
4. Dữ liệu thu được được đóng gói theo định dạng JSON và chuyển đến hệ thống giám sát.

3.3.2.d Giao thức MQTT

Giao thức MQTT được sử dụng để truyền dữ liệu từ ESP32 lên nền tảng CoreIoT. MQTT hoạt động theo mô hình publish/subscribe, cho phép truyền dữ liệu nhẹ, ổn định và phù hợp với các thiết bị IoT có tài nguyên hạn chế.

3.3.2.1 Tầng truyền tải MQTT trong hệ thống được triển khai dựa trên kết nối TCP/IP và mạng WiFi. Vi điều khiển duy trì kết nối liên tục với máy chủ MQTT để đảm bảo việc truyền dữ liệu thời gian thực và nhận lệnh điều khiển từ hệ thống.

Triển khai trong hệ thống MQTT được sử dụng cho ba loại dữ liệu chính:

- **Telemetry:** truyền dữ liệu cảm biến như NO2, PM2.5, PM10 nhiệt độ và độ ẩm.
- **Thuộc tính thiết bị:** gửi hoặc cập nhật thông tin cấu hình.
- **Điều khiển từ xa (RPC):** nhận lệnh và phản hồi từ nền tảng CoreIoT.

Các dữ liệu được gửi theo định dạng JSON, đảm bảo dễ mở rộng và tương thích với các hệ thống giám sát hiện đại.

Ưu điểm của MQTT

- Tối ưu băng thông và độ trễ thấp.
- Đảm bảo độ tin cậy và khả năng duy trì kết nối.
- Hỗ trợ truyền thông hai chiều giữa thiết bị và máy chủ.

Nhờ sử dụng MQTT, hệ thống có thể truyền dữ liệu ổn định, liên tục và xử lý điều khiển từ xa một cách hiệu quả.

3.3.2.e Giao thức HTTP

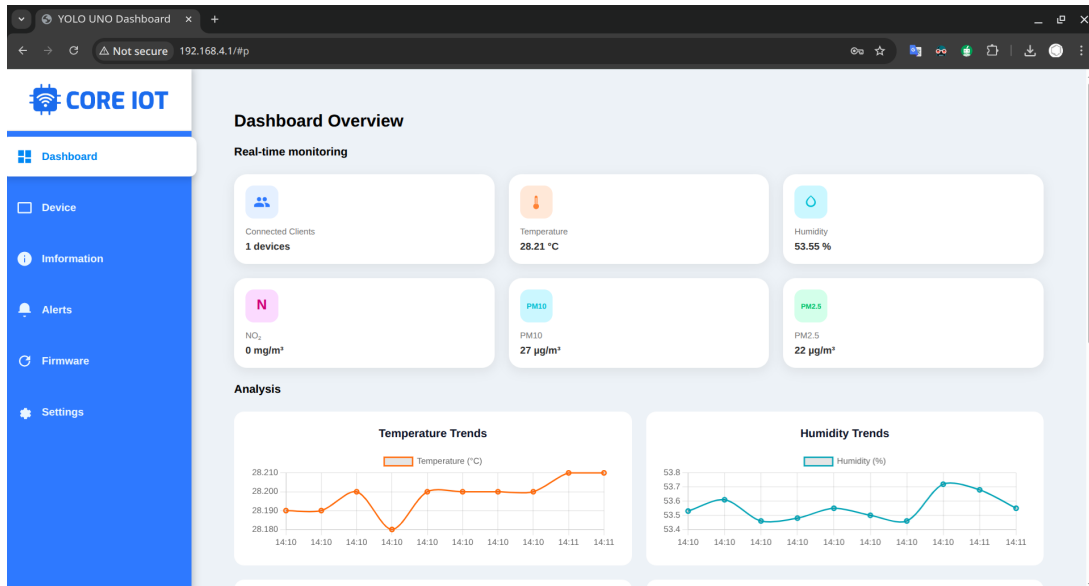
HTTP (Hypertext Transfer Protocol): Được sử dụng để triển khai WebServer cấu hình cục bộ. Khi ESP32 ở chế độ Access Point (AP Mode), người dùng truy cập địa chỉ `http://192.168.4.1` để thiết lập thông số Wi-Fi, MQTT và cấu hình thiết bị.

3.4 Thiết kế giao diện

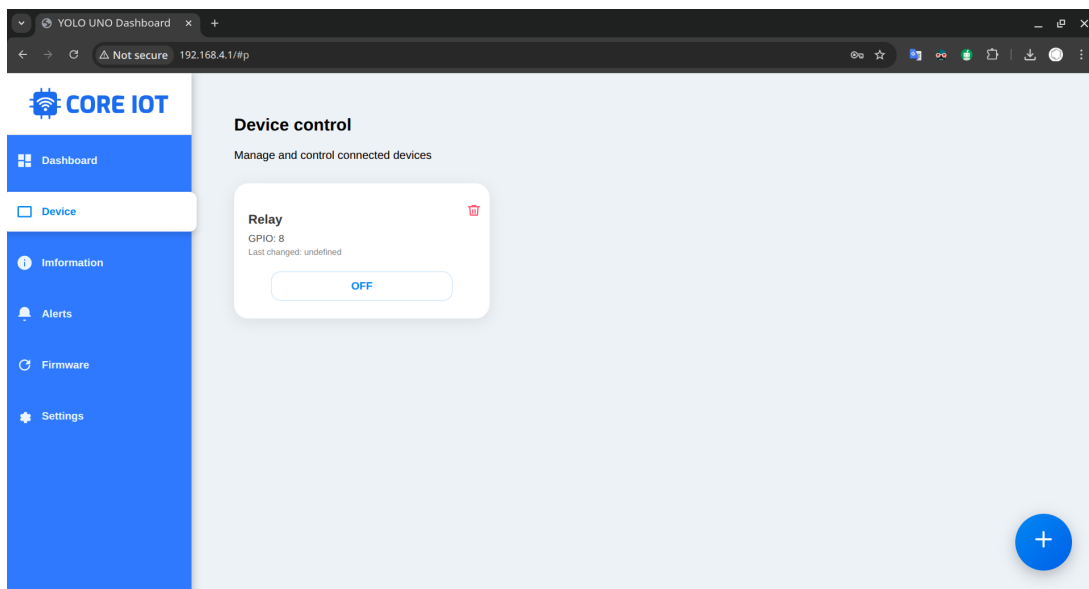
3.4.1 Giao diện Webserver trên ESP32

Giao diện Webserver được triển khai trên ESP32 giúp giám sát và cấu hình hệ thống lúc khởi tạo thông qua mạng nội bộ.

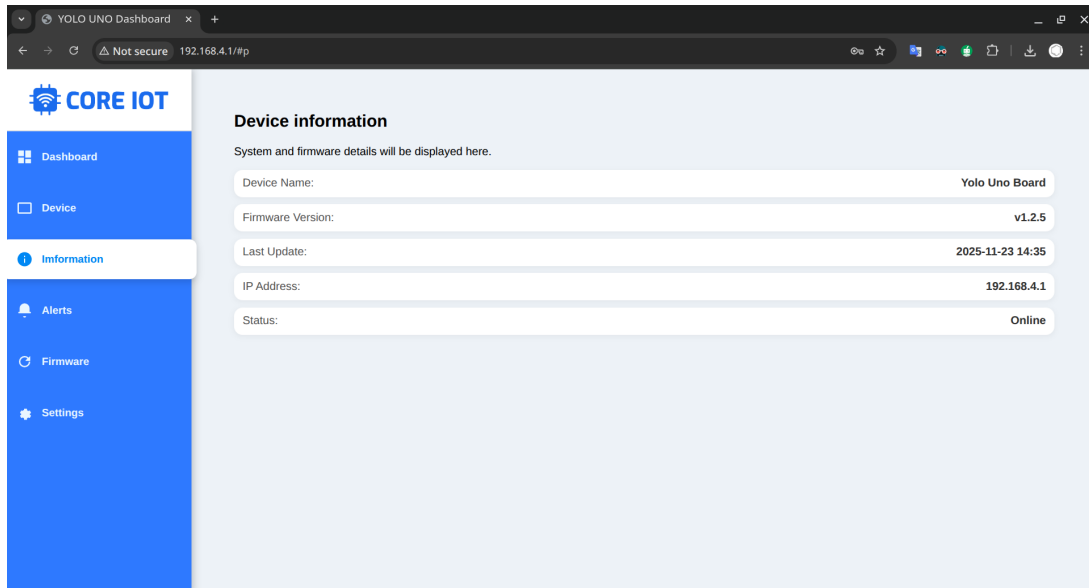
- **Dashboard:** Hiển thị dữ liệu cảm biến theo thời gian thực với cơ chế tự động cập nhật mà không cần tải lại trang.



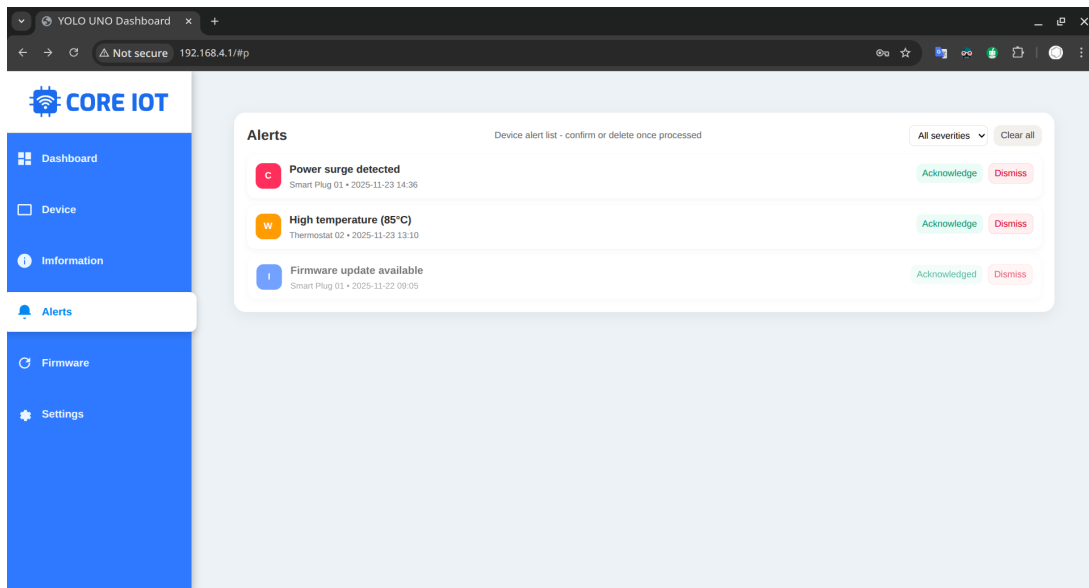
- **Device:** Trang điều khiển bật tắt hoặc tạo mới cấu hình cho các thiết bị.



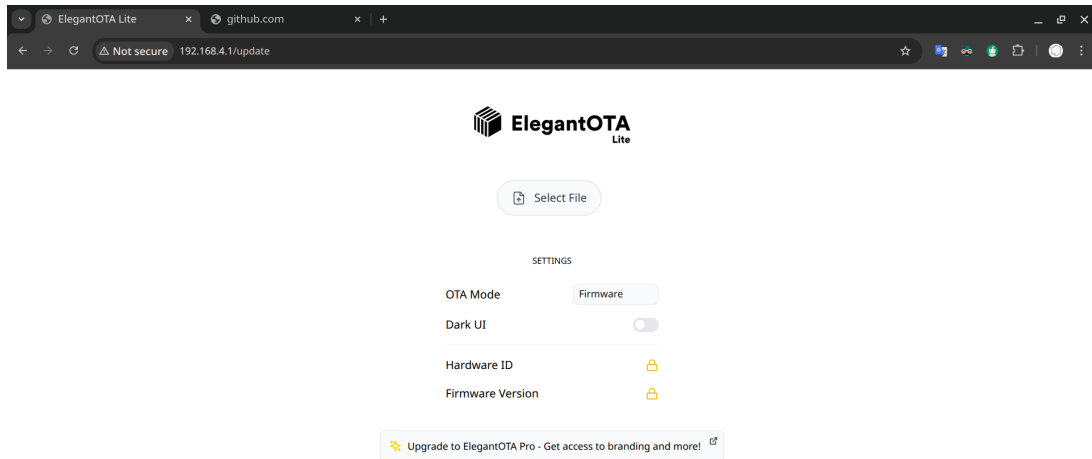
- **Information:** Hiển thị thông tin mô tả hệ thống gồm phiên bản firmware, phần cứng và tài liệu liên quan.



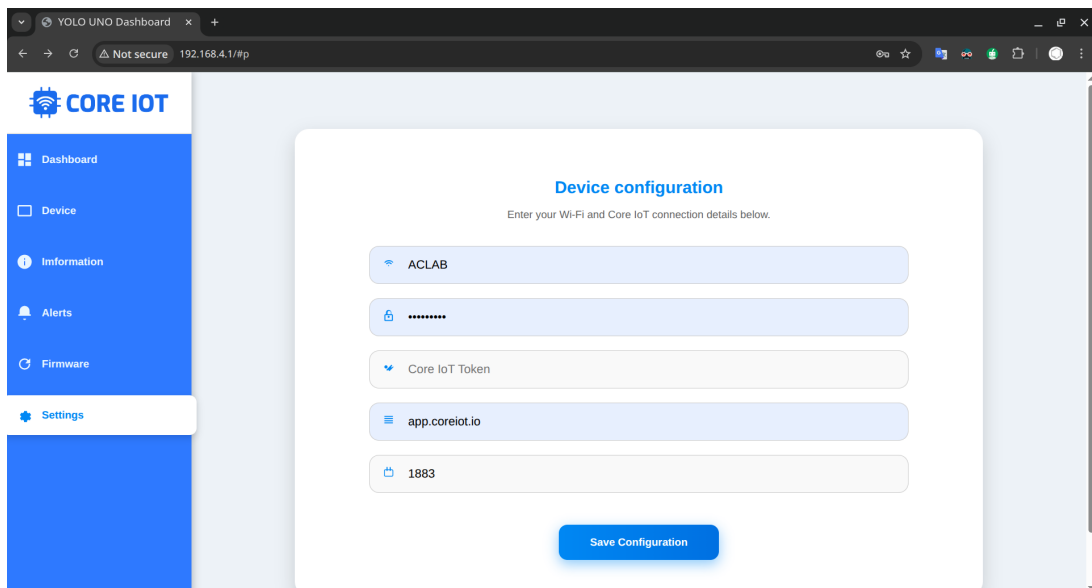
- **Alerts:** Danh sách các cảnh báo sinh ra khi thông số vượt ngưỡng cài đặt, bao gồm loại cảnh báo, thời điểm và trạng thái xử lý.



- **Firmware Update:** Cho phép cập nhật firmware OTA bằng cách tải tệp .bin trực tiếp từ trình duyệt.



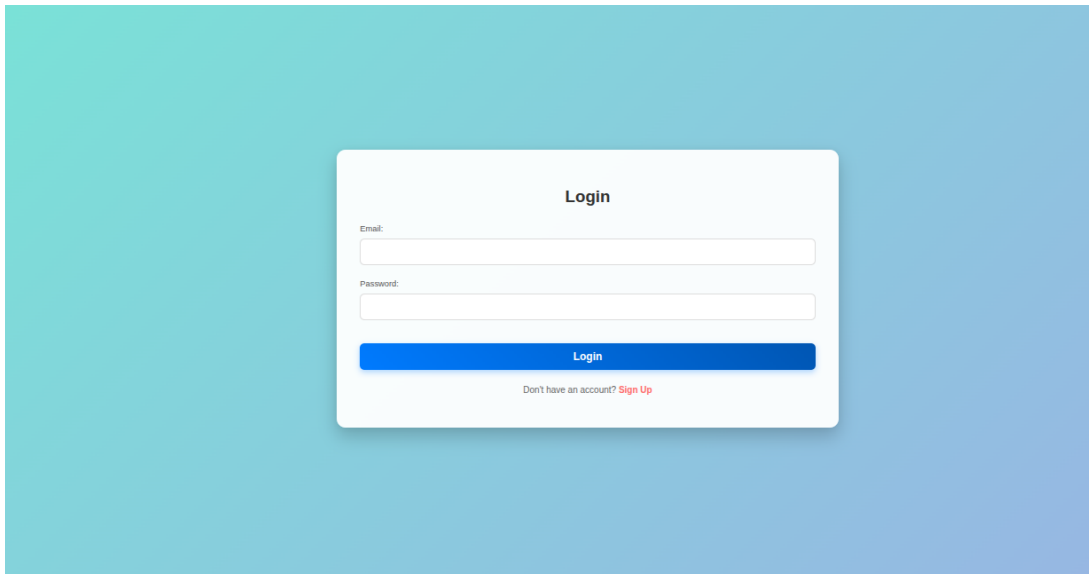
- **Settings:** Trang cấu hình hệ thống như Wi-Fi, thông tin MQTT/HTTP server, token, chu kỳ cập nhật dữ liệu và các ngưỡng cảnh báo. Các tham số được lưu trong flash để duy trì sau khi thiết bị khởi động lại.



3.4.2 Giao diện Website

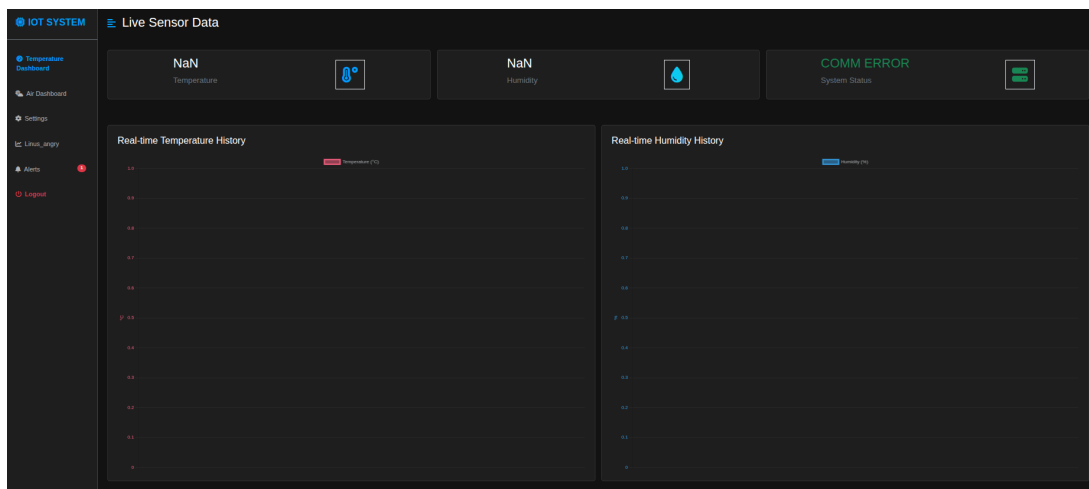
Giao diện Website được xây dựng nhằm cung cấp một bảng điều khiển tập trung, thân thiện với người dùng, cho phép giám sát dữ liệu cảm biến, quản lý cấu hình thiết bị và nhận các cảnh báo theo thời gian thực. Giao diện được thiết kế để hoạt động tốt trên các thiết bị di động (responsive design) và sử dụng các thư viện giao diện hiện đại như Bootstrap.

- **Màn hình Đăng nhập (Login):** Cung cấp giao diện để người dùng xác thực danh tính bằng Email và Mật khẩu. Khi đăng nhập thành công, hệ thống sẽ cấp một Access Token để đảm bảo tính bảo mật và phân quyền truy cập dữ liệu cá nhân của người dùng.



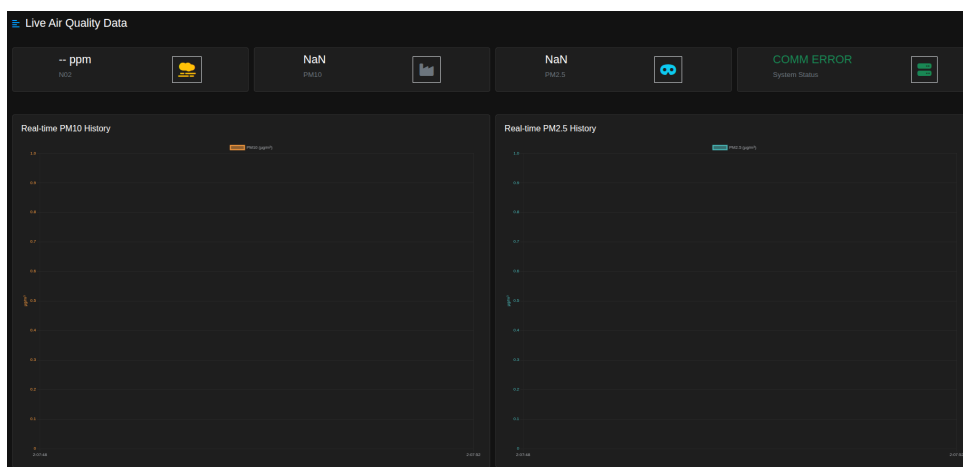
Hình 5: Màn hình Đăng nhập (Login Screen)

- **Dashboard Giám sát Nhiệt độ và Độ ẩm:** Dashboard chính hiển thị dữ liệu nhiệt độ và độ ẩm được truy xuất từ cơ sở dữ liệu (Database) theo ID người dùng. Dữ liệu được trình bày dưới dạng biểu đồ đường (line chart) để theo dõi sự thay đổi theo thời gian.



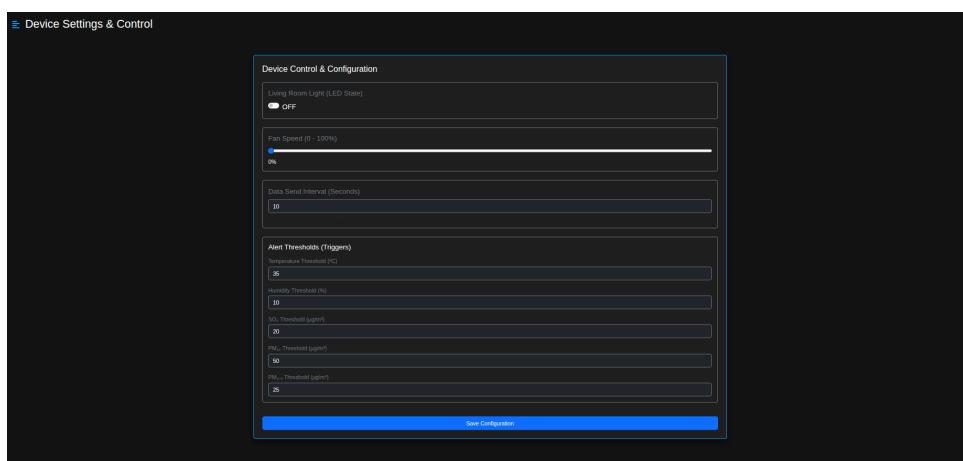
Hình 6: Dashboard nhiệt độ và độ ẩm trên Website

- **Dashboard Giám sát Chất lượng Không khí:** Hiển thị các thông số chất lượng không khí như $PM_{2.5}$, PM_{10} , và SO_2 (hoặc NO_2). Tương tự, dữ liệu được lọc theo người dùng và hiển thị bằng các biểu đồ trực quan.



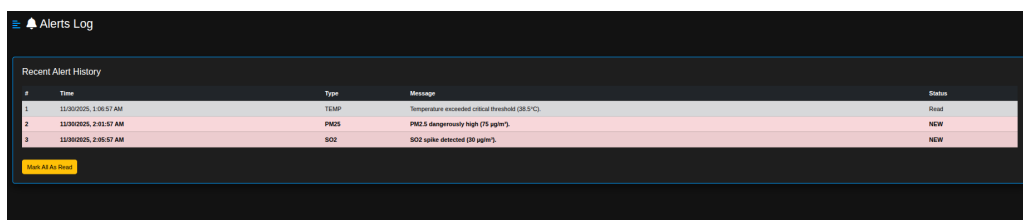
Hình 7: Dashboard chất lượng không khí trên Website

- **Trang Cấu hình và Điều khiển Thiết bị (Settings):** Cho phép người dùng thiết lập các thông số hoạt động của thiết bị như tốc độ quạt (Fan Speed), trạng thái đèn (LED State), khoảng thời gian gửi dữ liệu (Data Interval Time). Quan trọng hơn, đây là nơi người dùng đặt các **ngưỡng cảnh báo** (Temperature Threshold, $PM_{2.5}$ Threshold, v.v.).



Hình 8: Trang Cấu hình và Điều khiển Thiết bị

- **Trang Lịch sử Cảnh báo (Alerts Log):** Hiển thị danh sách các cảnh báo (Alerts) đã xảy ra do dữ liệu vượt ngưỡng. Giao diện này hiển thị thời gian, loại cảnh báo, nội dung thông báo, và trạng thái (Đã đọc/Chưa đọc). Thanh Sidebar (Menu) hiển thị số lượng cảnh báo chưa đọc theo thời gian thực.



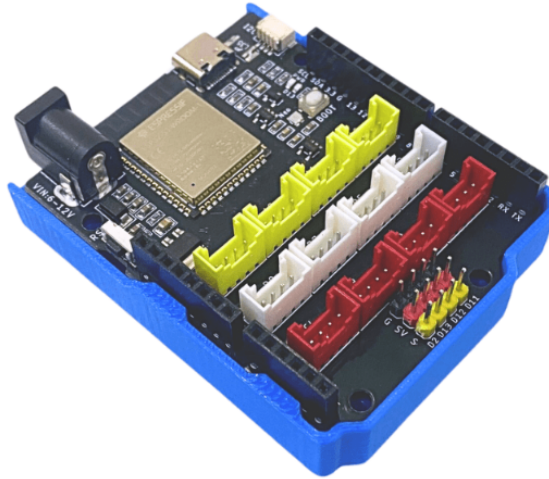
#	Time	Type	Message	Status
1	11/30/2025, 1:06:57 AM	TEMP	Temperature exceeded critical threshold (38.5°C).	Read
2	11/30/2025, 2:01:57 AM	PM25	PM2.5 dangerously high (75 µg/m³).	NEW
3	11/30/2025, 2:05:57 AM	SO2	SO2 spike detected (30 µg/m³).	NEW

Hình 9: Trang Lịch sử Cảnh báo

4 Triển khai

4.1 Phần cứng sử dụng

Board Yolo UNO



Hình 10: Board Yolo UNO

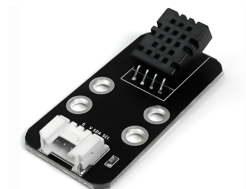
Yolo UNO là mạch lập trình được OhStem thiết kế theo form của Arduino Uno, sử dụng vi điều khiển ESP32-S3 là dòng chip mới của Espressif, với bộ nhớ Flash lên đến 16MB và bộ nhớ PSRAM 8MB. Nhờ đó, bo mạch đáp ứng tốt các ứng dụng IoT/AI cần nhiều tài nguyên bộ nhớ, đồng thời vẫn giữ được kích thước nhỏ gọn và cách bố trí tương thích với Arduino Uno truyền thống.[1]

Ngoài kích thước và sơ đồ chân tương thích hoàn toàn với Arduino Uno, Yolo UNO còn được tích hợp sẵn 12 cổng kết nối chuẩn Grove, giúp dễ dàng mở rộng với các module và cảm biến trong hệ sinh thái thiết bị AI/IoT của OhStem. Một số đặc tả kỹ thuật quan trọng của bo mạch như sau:

- **Vi điều khiển:** ESP32-S3 với CPU dual-core Tensilica LX7 xung nhịp tối đa 240 MHz, bộ nhớ ngoài 16 MB Flash và 8 MB PSRAM. Vi điều khiển tích hợp bộ tăng tốc xử lý tín hiệu (vector instructions) và *native USB OTG*, phù hợp cho các bài toán IoT, xử lý tín hiệu và ứng dụng AI nhẹ.[2]
- **Kết nối không dây:**
 - WiFi băng tần 2.4 GHz, hỗ trợ chuẩn 802.11b/g/n, bảo mật WPA/WPA2, thích hợp cho kết nối mạng cục bộ và đám mây.
 - Bluetooth 5 với BLE và Mesh, cho phép kết nối với cảm biến, thiết bị ngoại vi công suất thấp hoặc xây dựng mạng lưới thiết bị không dây.
- **GPIO và ngoại vi số:**
 - Các chân số được đưa ra theo form Arduino Uno (Dx, Ax), dễ dàng tương thích với shield Arduino hiện có.
 - Hỗ trợ đa chức năng trên mỗi chân: GPIO số, PWM điều khiển động cơ/LED, I²C, SPI, UART..., giúp linh hoạt trong thiết kế phần cứng.
 - 4 chân cắm dạng GVS dành riêng cho điều khiển động cơ Servo, thuận tiện cho các bài toán robot và cơ cấu chấp hành.
- **ADC, UART và các giao tiếp ngoại vi khác:**
 - Nhiều kênh ADC độ phân giải tới 12 bit, được ánh xạ ra các chân Analog, phù hợp đo cảm biến nhiệt độ, ánh sáng, cảm biến môi trường,...

- Tối đa 3 UART phần cứng (UART0/1/2) của ESP32-S3, trong đó một UART thường dùng cho giao tiếp USB-Serial, các UART còn lại có thể sử dụng để giao tiếp với module GPS, module GSM, cảm biến khoảng cách,...
- Hỗ trợ đầy đủ các bus I²C và SPI để kết nối với màn hình, cảm biến IMU, bộ nhớ ngoài hoặc các module mở rộng khác.
- **Cổng kết nối Grove và mở rộng I²C:**
 - 12 cổng kết nối chuẩn Grove, bao gồm: cổng Analog, Digital và I²C, cho phép cắm nhanh các module/cảm biến mà không cần hàn.
 - Cổng I²C chuẩn STEMMA QT/QWIIC giúp dễ dàng nối chuỗi nhiều module I²C trên cùng một bus.
- **Cấp nguồn và chỉ thị trạng thái:**
 - Cấp nguồn qua cổng USB Type-C hoặc jack DC 5521 (tối đa 12 V), thuận tiện cho cả môi trường phòng lab và ngoài thực địa.
 - Tích hợp đèn LED báo nguồn, LED trên chân D13 và LED RGB NeoPixel, hỗ trợ hiển thị trạng thái hệ thống hoặc hiệu ứng ánh sáng đơn giản.
- **Hỗ trợ lập trình:**
 - **Arduino:** Hỗ trợ đầy đủ trong Arduino IDE sau khi cài đặt board ESP32, thích hợp cho sinh viên và người mới bắt đầu.
 - **PlatformIO:** Có thể phát triển firmware trên nền tảng PlatformIO (VS Code), hỗ trợ cả framework Arduino lẫn ESP-IDF, thuận tiện cho quản lý thư viện, build và debug.
 - **ESP-IDF:** Hỗ trợ lập trình trực tiếp bằng bộ SDK chính thức của Espressif, phù hợp cho các ứng dụng cần tối ưu hiệu năng và tận dụng tối đa tài nguyên phần cứng.

Cảm biến DHT20

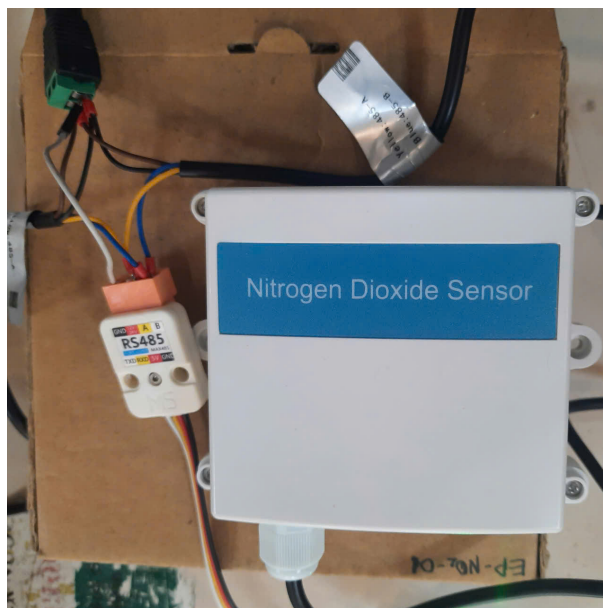


Hình 11: Cảm biến DHT20

DHT20 là cảm biến nhiệt độ và độ ẩm số, tích hợp sẵn chip xử lý và cảm biến độ ẩm dung kháng, xuất dữ liệu dạng số thông qua giao tiếp I²C[3]. Nhờ đó, việc giao tiếp với vi điều khiển trên board Yolo UNO trở nên đơn giản, ổn định và phù hợp cho các ứng dụng giám sát môi trường và hệ thống IoT.[4]

- **Dải đo và độ chính xác:**
 - Độ ẩm tương đối: 0–100 %RH, độ chính xác điển hình khoảng $\pm 3\%$ RH.
 - Nhiệt độ: từ -40°C đến 80°C , độ chính xác điển hình khoảng $\pm 0.5^{\circ}\text{C}$.
- **Nguồn nuôi và tiêu thụ năng lượng:**
 - Điện áp hoạt động: 2.2 V–5.5 VDC, có thể cấp trực tiếp từ 3.3 V hoặc 5 V của Yolo UNO.
 - Dòng tiêu thụ thấp, phù hợp cho các ứng dụng cần tiết kiệm năng lượng.
- **Giao tiếp và cách sử dụng:**
 - Giao tiếp qua bus I²C với địa chỉ mặc định 0x38, dễ dàng kết nối với ESP32-S3 thông qua các chân I²C hoặc cổng Grove.
 - Dữ liệu nhiệt độ và độ ẩm đã được xử lý và hiệu chuẩn sẵn bên trong, vi điều khiển chỉ cần gửi lệnh đo và đọc về gói dữ liệu số.

Cảm biến nồng độ NO_2 trong không khí



Hình 12: Cảm biến nồng độ NO_2 trong không khí

Cảm biến nồng độ NO_2 được sử dụng để đo và giám sát mức khí Nitrogen Dioxide trong môi trường không khí. Đây là loại cảm biến công nghiệp sử dụng giao thức truyền thông Modbus RTU qua RS485, cho phép truyền dữ liệu ổn định trên khoảng cách xa và dễ dàng tích hợp vào hệ thống IoT. Cảm biến có dải đo rộng, thời gian đáp ứng nhanh và độ chính xác cao, phù hợp cho các ứng dụng giám sát môi trường trong nhà và ngoài trời.

Thông số kỹ thuật:

- Điện áp hoạt động: 10–30VDC
- Tín hiệu ngõ ra: RS485 (Modbus RTU)
- Nhiệt độ làm việc: -20°C đến 50°C
- Độ ẩm làm việc: 15%–90%RH
- Độ phân giải NO_2 :
 - 20ppm: 0.1ppm
 - 2000ppm: 1ppm
- Thời gian đáp ứng:
 - 20ppm: $\leq 30\text{s}$
 - 2000ppm: $\leq 60\text{s}$
- Thời gian làm nóng: ≥ 5 phút
- Độ chính xác: $\pm 5\%\text{FS}$

Cảm biến bụi PM2.5 và PM10



Hình 13: Cảm biến bụi PM2.5 và PM10

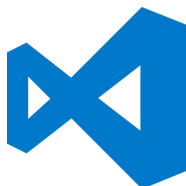
Cảm biến bụi được sử dụng để đo nồng độ hạt bụi lơ lửng trong không khí, tập trung vào hai chỉ số quan trọng là PM2.5 và PM10. Đây là cảm biến công nghiệp dùng giao thức truyền thông RS485 Modbus RTU, cho phép truyền dữ liệu ổn định trên khoảng cách xa và dễ dàng tích hợp vào các hệ thống giám sát chất lượng không khí. Cảm biến phù hợp cho ứng dụng trong nhà, ngoài trời, khu dân cư, nhà máy hoặc các hệ thống cảnh báo ô nhiễm môi trường.

Thông số kỹ thuật:

- Nguồn cấp: 10–30VDC
- Công suất: 0.5W
- Nhiệt độ làm việc: -20°C đến 60°C
- Độ ẩm làm việc: 0–95%RH (không ngưng tụ)
- Dải đo bụi:
 - PM2.5: $0\text{--}1000\text{ }\mu\text{g}/\text{m}^3$
 - PM10: $0\text{--}1000\text{ }\mu\text{g}/\text{m}^3$
- Hiệu suất đếm hạt: 50% @ $0.3\text{ }\mu\text{m}$, 98% @ $\geq 0.5\text{ }\mu\text{m}$
- Độ chính xác PM2.5: $\pm 3\%\text{FS}$ (tại $100\text{ }\mu\text{g}/\text{m}^3$, 25°C , 50%RH)
- Độ phân giải: $1\text{ }\mu\text{g}/\text{m}^3$ cho PM2.5 và PM10
- Tốc độ phản hồi: $\leq 90\text{s}$
- Thời gian làm nóng: ≤ 2 phút
- Phương pháp lắp đặt: Treo tường
- Tín hiệu đầu ra: RS485 (Modbus RTU)

4.2 Công cụ phát triển

Visual Studio Code



Hình 14: Visual Studio Code

Trong bài tập lớn này, Visual Studio Code (VS Code) được sử dụng làm môi trường soạn thảo mã nguồn chính. VS Code cung cấp giao diện trực quan, hỗ trợ nhiều ngôn ngữ lập trình, tích hợp Git và hệ thống mở rộng phong phú, giúp việc viết mã, quản lý phiên bản và tổ chức cấu trúc dự án trở nên thuận tiện.

Vai trò chính của VS Code:

- Là IDE trung tâm để chỉnh sửa mã nguồn, cấu hình dự án và xem log/console.
- Hỗ trợ cài đặt và quản lý các tiện ích mở rộng (extension) phục vụ phát triển nhúng, như PlatformIO, các công cụ định dạng/lint mã và hỗ trợ ngôn ngữ C/C++.
- Tạo môi trường làm việc thống nhất cho toàn bộ nhóm, dễ chia sẻ và tái sử dụng cấu hình.

PlatformIO



Hình 15: PlatformIO

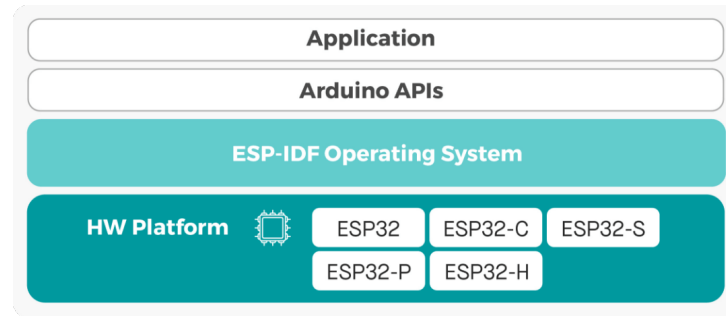
PlatformIO là phần mở rộng được tích hợp vào VS Code, đóng vai trò là nền tảng phát triển nhúng cho board Yolo UNO (ESP32-S3). Thay vì phải tự cài đặt toolchain, thư viện và cấu hình build thủ công, PlatformIO cung cấp một hệ sinh thái tự động hóa các bước này.

Các chức năng chính của PlatformIO:

- Quản lý cấu trúc dự án nhúng theo từng *environment* (ví dụ: cấu hình build cho các board ESP32 khác).
- Tự động cài đặt và quản lý thư viện, toolchain và cấu hình build phù hợp với nền tảng ESP32.
- Cung cấp các tác vụ *build*, *upload* firmware xuống board và mở Serial Monitor trực tiếp trong VS Code.
- Hỗ trợ nhiều framework như Arduino và ESP-IDF.

Arduino framework cho ESP32

Arduino framework cho ESP32 cung cấp một lớp trừu tượng phần cứng đơn giản, cho phép lập trình board Yolo UNO bằng cú pháp Arduino quen thuộc (hàm `setup()`, `loop()`, các hàm `digitalWrite()`, `analogRead()`, ...).[5]



Hình 16: Mô hình hoạt động của Arduino framework cho ESP32

Để giảm độ phức tạp khi phát triển bài tập lớn, nhóm lựa chọn sử dụng framework này vì có sẵn rất nhiều thư viện hỗ trợ cho cảm biến, kết nối mạng và giao tiếp ngoại vi, đồng thời cộng đồng người dùng đông đảo và tài liệu tham khảo phong phú. Framework được PlatformIO tự động cài đặt và cấu hình khi khai báo `platform = espressif32` và `framework = arduino` trong file `platformio.ini`.

Vai trò và tính năng chính:

- Đơn giản hóa việc cấu hình và điều khiển GPIO, UART, I²C, SPI trên ESP32 thông qua các API Arduino.
- Hỗ trợ nhiều thư viện sẵn có cho WiFi, MQTT, HTTP, cũng như các thư viện cho cảm biến DHT20, cảm biến NO2, PM2.5, PM10, ...
- Giảm độ phức tạp khi bắt đầu phát triển trên ESP32, đồng thời vẫn tận dụng được sức mạnh phần cứng của vi điều khiển.

Ngoài ra còn có các toolchain và compiler do PlatformIO tự động cung cấp như Xtensa toolchain (gcc, g++, linker), Python, esptool.py để flash firmware, ...

FreeRTOS



Hình 17: FreeRTOS

Nhóm định hướng phát triển hệ thống theo mô hình đa nhiệm, trong đó các chức năng như đọc cảm biến, xử lý dữ liệu, truyền thông và quản lý trạng thái thiết bị được tách riêng thành từng phần độc lập để đảm bảo hoạt động ổn định và dễ mở rộng. Vì vậy, nhóm quyết định sử dụng thư viện FreeRTOS được tích hợp sẵn trong Arduino framework dành cho ESP32.

FreeRTOS là một hệ điều hành thời gian thực nhẹ, được tích hợp mặc định trong SDK của ESP32 và có thể truy cập trực tiếp thông qua Arduino. Việc sử dụng FreeRTOS giúp nhóm dễ dàng tạo và quản lý các tác vụ (task), đồng bộ dữ liệu bằng *semaphore* hoặc *queue*, đồng thời cho phép các chức năng chạy song song mà không làm ảnh hưởng đến hiệu năng tổng thể của hệ thống.[6]

Các lý do chính khi lựa chọn FreeRTOS:

- Hỗ trợ đa nhiệm thực sự, phù hợp với yêu cầu chia tách chức năng (đọc cảm biến, xử lý dữ liệu, truyền thông, quản lý trạng thái thiết bị).
- Được tích hợp sẵn trong framework nên không cần cài đặt hoặc cấu hình phức tạp.
- Được tối ưu cho ESP32 và được cộng đồng sử dụng rộng rãi, giúp tăng độ tin cậy và tính ổn định của hệ thống.
- Thuận tiện cho việc mở rộng và bảo trì mã nguồn trong tương lai nhờ kiến trúc modular theo từng tác vụ.

Nhờ sử dụng FreeRTOS, hệ thống có thể được triển khai theo hướng mô-đun, trong đó mỗi tác vụ đảm nhiệm một chức năng riêng và hoạt động song song một cách hiệu quả, giúp dễ dàng theo dõi, gỡ lỗi và nâng cấp trong các phiên bản sau này.

4.3 Các đoạn mã chính

Hệ thống được phát triển dựa trên mô hình đa nhiệm, sử dụng FreeRTOS để quản lý các tác vụ. Do đó, các đoạn mã chính sẽ được viết dưới dạng các tác vụ (task) trong FreeRTOS.

4.3.1 Global

Module **Global** dùng để tập trung các thành phần được sử dụng chung trong toàn hệ thống, giúp mã nguồn thống nhất và dễ quản lý. Hai tệp `global.h` và `global.cpp` lần lượt đảm nhiệm vai trò khai báo và định nghĩa các tài nguyên này.

- **Thư viện dùng chung:** các thư viện nền tảng và thư viện hệ điều hành thời gian thực (ví dụ: `Arduino.h`, `FreeRTOS.h`, `task.h`).
- **Định nghĩa cấu hình chung:** hằng số cấu hình và tham số hệ thống (ví dụ: tốc độ UART, thời gian chu kỳ, ưu tiên của task).
- **Định nghĩa phần cứng:** khai báo các chân GPIO và kết nối phần cứng được sử dụng trong hệ thống (ví dụ: chân điều khiển LED, cảm biến, relay).
- **Biến toàn cục:** các biến trạng thái và cấu trúc dữ liệu dùng chung cho nhiều module (ví dụ: biến trạng thái hệ thống, cấu trúc chứa giá trị cảm biến).
- **Tài nguyên đồng bộ FreeRTOS:** các queue, semaphore hoặc mutex để chia sẻ dữ liệu an toàn giữa các task (ví dụ: `QueueHandle_t sensorQueue`, `SemaphoreHandle_t dataMutex`).

Nhờ việc tập trung các phần tử dùng chung vào module Global, các task trong FreeRTOS có thể giao tiếp và đồng bộ một cách rõ ràng, đồng thời hạn chế xung đột trong quá trình truy cập tài nguyên.

4.3.2 Main

Mã giả `main.cpp`:

```
1  IMPORT configuration files and functional modules
2
3  FUNCTION setup():
4      Initialize Serial communication with baud rate 115200
5      Initialize I2C communication with configured SDA and SCL pins
6
7      Create FreeRTOS tasks:
8          webserver processing task
9          mqtt client connection task
10         DHT sensor reading task
11         AIR sensor reading task
12         Relay control task
13         NeoPixel LED control task
14         ...
15  FUNCTION loop():
16      No operation
```

File mã chính `main.cpp` đóng vai trò điểm khởi tạo của toàn bộ chương trình. File này thực hiện các cấu hình ban đầu cho hệ thống và tạo các tác vụ FreeRTOS để vận hành các chức năng chính.

- **Khởi tạo hệ thống:** thiết lập các giao tiếp ngoại vi cần thiết như `Serial` cho debug, `UART` cho cảm biến, `I2C` cho cảm biến, ...

- **Tạo các tác vụ FreeRTOS:** mỗi tác vụ đảm nhiệm một chức năng độc lập trong hệ thống, giúp chương trình vận hành theo mô hình đa nhiệm.
- **Vòng lặp chính:** hàm `loop()` được giữ trống vì toàn bộ logic được chuyển sang các task FreeRTOS thay cho việc chạy tuần tự theo vòng lặp của Arduino.

Nhờ tổ chức theo dạng đa nhiệm, `main.cpp` chỉ tập trung vào thiết lập tài nguyên và kích hoạt các task, trong khi các chức năng được phân tách rõ ràng và vận hành song song thông qua FreeRTOS.

4.3.3 Task Main Server

```
1 FUNCTION main_server_task:
2     Configure BOOT button pin as input
3     Start Access Point mode
4     Initialize web server
5
6     LOOP forever:
7
8         Handle incoming HTTP client requests
9
10        -- Check BOOT button to switch to AP mode --
11        IF BOOT button is pressed:
12            Wait briefly and confirm press
13            Read current AP mode state
14            IF system is not in AP mode:
15                Restart AP mode
16                Reinitialize web server
17            END IF
18        END IF
19
20        -- Check STA (WiFi station) connection process --
21        IF system is attempting STA connection:
22            IF WiFi is connected:
23                Print STA IP address
24                Mark WiFi as connected
25                Signal Internet availability
26                Set AP mode flag to false
27                Set connecting flag to false
28            ELSE IF connection timeout (10 seconds):
29                Print failure message
30                Return to AP mode
31                Reinitialize server
32                Clear connecting and WiFi-connected flags
33            END IF
34        END IF
35    END LOOP
36 END FUNCTION
```

Tác vụ `main_server_task` chịu trách nhiệm quản lý chế độ mạng (AP/STA), xử lý nút BOOT, và vận hành webserver. Hành vi chính của tác vụ gồm:

- Khởi động hệ thống ở chế độ **AP mode** và cấu hình webserver.
- Liên tục xử lý các yêu cầu từ client thông qua hàm `server.handleClient()`.
- Giám sát nút BOOT; khi nhấn giữ, hệ thống sẽ chuyển sang chế độ AP để cho phép cấu hình lại WiFi (nếu chưa ở AP mode).
- Theo dõi trạng thái kết nối WiFi ở chế độ **STA mode**:

- Nếu kết nối thành công, cập nhật cờ trạng thái WiFi, thông báo IP và kích hoạt semaphores báo hiệu có Internet.
- Nếu kết nối thất bại sau 10 giây, tự động quay lại chế độ AP và khởi động lại webserver.

4.3.4 Task CoreIoT

```
1 FUNCTION coreiot_task:
2     Wait until the Internet is connected
3
4     LOOP:
5         IF ThingsBoard client is not connected:
6
7             IF connection fails:
8                 Wait 5 seconds
9                 try again after 5 seconds
10            END IF
11
12            IF services APIs are not subscribed:
13                subscribe to services APIs:
14                    rpc, attribute update, OTA update
15            END IF
16        END IF
17
18        -- Periodic telemetry sending --
19        IF telemetry period is reached:
20            send telemetry data to Server
21        END IF
22
23    END LOOP
24
25 END FUNCTION
```

4.3.5 Task DHT20

Mã giả dht_task.cpp:

```
1 FUNCTION dht_task():
2     DECLARE temperature, humidity
3     REPEAT
4         IF DHT20 is not connected
5             PRINT "Disconnected, retrying..."
6             WAIT 5 seconds
7             CONTINUE
8         END IF
9         WAIT for SENSOR_TRIGGER event
10        READ DHT20 sensor
11        IF reading fails
12            SET temperature, humidity to -1
13        ELSE
14            GET temperature, humidity from DHT20
15        END IF
16
17        FORMAT temperature, humidity as JSON
18        SEND data to queue
19
20        UPDATE global temperature, humidity
```

```
21     END REPEAT
22 END FUNCTION
```

4.3.6 Task Air Sensor

Mã giả air_sensor_task.cpp:

```
1 FUNCTION air_sensor_task():
2     DECLARE no2, pm25, pm10
3     REPEAT
4         IF SENSOR is not connected
5             PRINT "Disconnected, retrying..."
6             WAIT 5 seconds
7             CONTINUE
8         END IF
9         WAIT for SENSOR_TRIGGER event
10        READ NO2 sensor
11        IF reading fails
12            SET no2, pm25, pm10 to 0
13        ELSE
14            GET no2, pm25, pm10 from SENSOR
15        END IF
16
17        FORMAT no2, pm25, pm10 as JSON
18        SEND data to queue
19
20    END REPEAT
21 END FUNCTION
```

Tác vụ air_sensor_task chịu trách nhiệm đọc nồng độ NO₂, PM2.5 và PM10 từ cảm biến Air Sensor theo chu kỳ. Quy trình hoạt động gồm:

- Kiểm tra kết nối cảm biến Air Sensor; nếu mất kết nối thì chờ và thử lại.
- Chờ tín hiệu yêu cầu đọc dữ liệu.
- Thực hiện đọc cảm biến Air Sensor; nếu lỗi thì gán giá trị mặc định, nếu thành công thì lấy nồng độ NO₂, PM2.5 và PM10 thực tế.
- Đóng gói dữ liệu vào chuỗi JSON và gửi vào queue.

4.3.7 Task Relay

Mã giả relay_task.cpp:

```
1 FUNCTION relay_task:
2     Initialize all relay pins as OUTPUT
3     Set all relays to OFF
4
5     LOOP forever:
6         Wait for incoming relay command from queue
7
8         IF received valid command:
9             IF target relay ID is valid:
10                Set relay state (ON or OFF)
11            ELSE:
12                Print "Invalid relay ID"
13            END IF
14        END IF
```

```
15   END LOOP
16   END FUNCTION
```

Tác vụ `relay_task` chịu trách nhiệm điều khiển các relay thông qua hàng đợi lệnh. Hoạt động của tác vụ gồm các bước:

- Khởi tạo tất cả chân điều khiển relay ở chế độ `OUTPUT` và đưa các relay về trạng thái `OFF`.
- Liên tục chờ lệnh điều khiển được gửi qua hàng đợi `xRelayControlQueue`.
- Khi nhận được lệnh:
 - Nếu ID relay hợp lệ, tác vụ bật hoặc tắt relay tương ứng.
 - Nếu ID không hợp lệ, in thông báo lỗi để hỗ trợ gỡ lỗi hệ thống.

Tác vụ chạy liên tục và đảm bảo các relay được điều khiển một cách an toàn và theo đúng yêu cầu từ hệ thống.

4.3.8 Các task khác

Ngoài các module chính trên còn một số task khác mà nhóm không làm chi tiết để tránh bài viết quá dài như điều khiển led tín hiệu, điều khiển led rgb, xử lý nút bấm ,...

4.4 Kết nối tới server

Core IoT (app.coreiot.io) là nền tảng IoT được xây dựng dựa trên **ThingsBoard**, một hệ thống mã nguồn mở phổ biến cho việc thu thập dữ liệu, hiển thị dashboard và quản lý thiết bị từ xa. ThingsBoard hỗ trợ nhiều giao thức như MQTT, HTTP và CoAP, đồng thời cung cấp khả năng mở rộng linh hoạt phục vụ các ứng dụng công nghiệp và dân dụng. Trong hệ thống của nhóm, Core IoT được sử dụng như *máy chủ dữ liệu IoT* làm nền tảng cho website dashboard của nhóm, cho phép lưu trữ, phân tích và điều khiển hệ thống thiết bị.



Hình 18: Core IoT



Hình 19: ThingsBoard

Thiết bị IoT (board Yolo UNO) của nhóm giao tiếp với Core IoT thông qua giao thức MQTT dựa trên tài liệu *ThingsBoard MQTT Device API*. Sau khi kết nối Internet, thiết bị thiết lập phiên MQTT bằng cách cung cấp địa chỉ máy chủ, cổng dịch vụ và token xác thực thiết bị.

Trong dự án này, hệ thống sử dụng đầy đủ các nhóm API bao gồm Telemetry API, Attribute API, Shared Attributes và RPC API, cùng với cơ chế OTA để cập nhật firmware từ xa.

Telemetry API Telemetry API cho phép thiết bị gửi dữ liệu thời gian thực (real-time data) lên máy chủ. API này được sử dụng để truyền các giá trị cảm biến như nhiệt độ môi trường, độ ẩm,...

Dữ liệu được đóng gói theo dạng JSON và gửi theo chu kỳ định sẵn. Máy chủ lưu trữ và hiển thị các giá trị này trên dashboard thời gian thực.

Attribute API Attribute API cho phép thiết bị gửi lên các thuộc tính không thay đổi liên tục, chẳng hạn SSID mạng WiFi, địa chỉ local IP, ...

Thuộc tính được gửi khi thiết bị khởi động hoặc khi có sự thay đổi. Các giá trị này giúp dashboard hiển thị thông tin hệ thống một cách đầy đủ và nhất quán.

RPC API RPC (Remote Procedure Call) là cơ chế điều khiển hai chiều giữa máy chủ và thiết bị. Máy chủ có thể gửi lệnh trực tiếp đến thiết bị, và thiết bị phản hồi lại theo thời gian thực.

Trong hệ thống, RPC API được dùng để bật/tắt relay, điều khiển GPIO, thực thi hành động điều khiển theo yêu cầu từ dashboard (đèn, bơm, quạt,...).

OTA Update Hệ thống cũng hỗ trợ cập nhật firmware từ xa bằng OTA (Over-The-Air). Thiết bị định kỳ kiểm tra phiên bản firmware trên máy chủ. Khi có bản cập nhật mới, firmware sẽ được tải về theo đường dẫn mà máy chủ cung cấp, kiểm tra tính toàn vẹn và tiến hành cập nhật.

Cơ chế OTA giúp giảm thiểu thời gian bảo trì và cho phép cập nhật hệ thống mà không cần thao tác thủ công trực tiếp trên thiết bị.

5 Kết quả và đánh giá

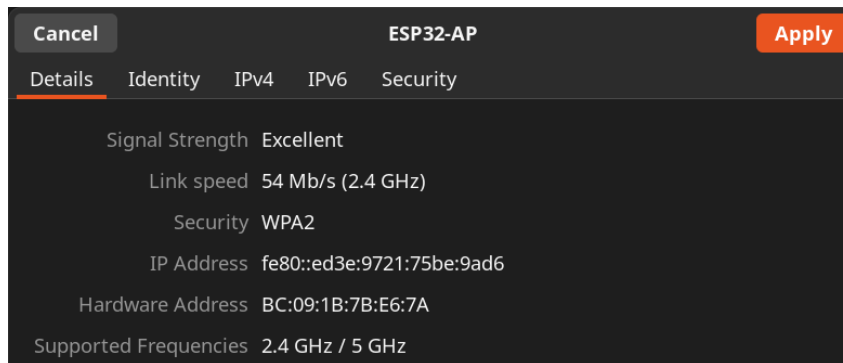
5.1 Kết quả

Nhóm tiến hành kết nối lắp đặt hệ thống theo như đã thiết kế:



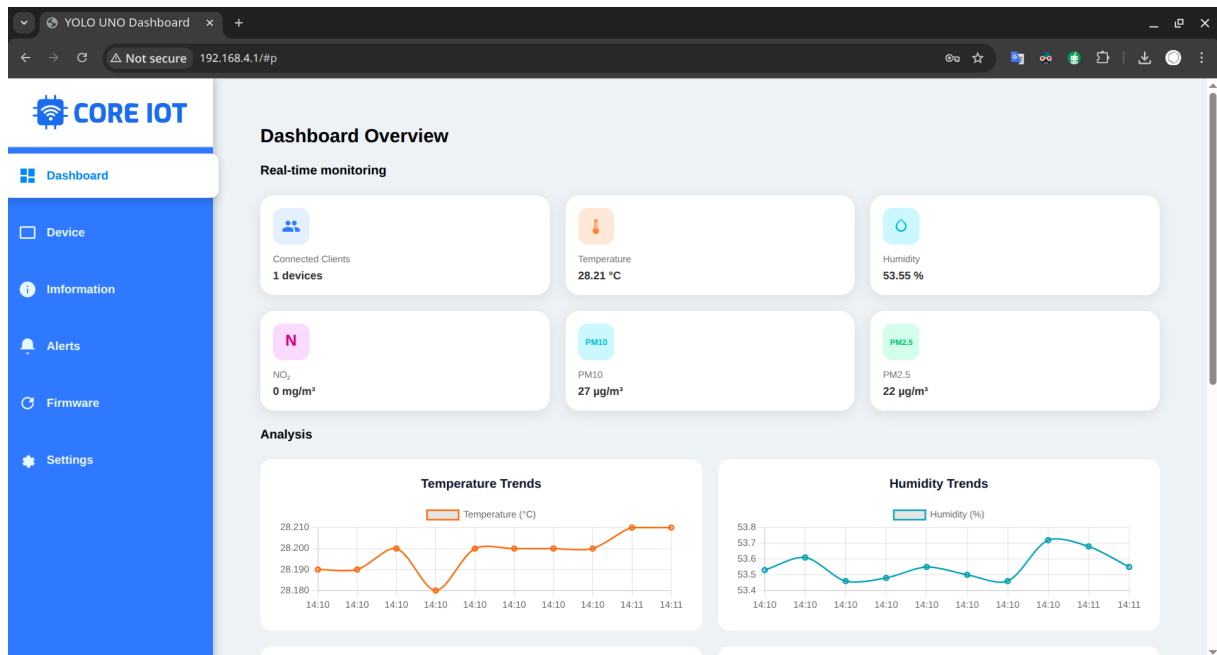
Hình 20: Hệ thống khi lắp đặt thực tế

Khi hệ thống khởi tạo, ESP32 hoạt động ở chế độ AP để phát ra WiFi và cấu hình webserver.



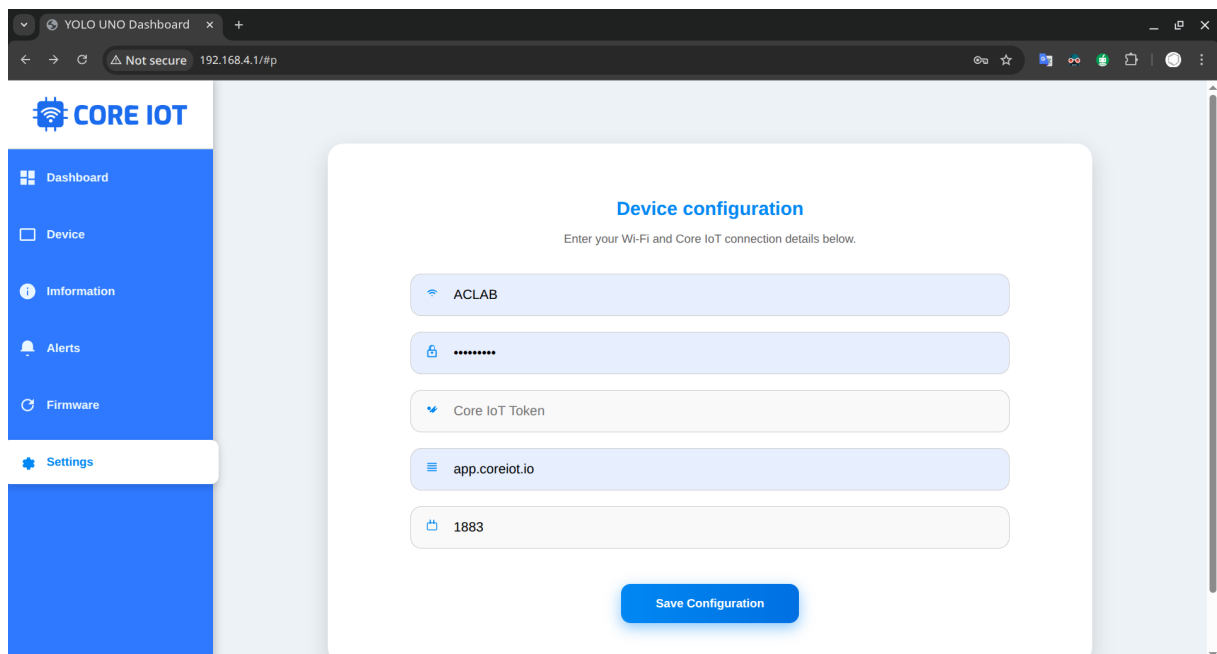
Hình 21: ESP32 hoạt động ở chế độ AP và phát ra WiFi

Sau khi kết nối thành công vào WiFi, Client truy cập tới địa chỉ 192.168.4.1 để truy cập tới dashboard của hệ thống để quan sát dữ liệu thu thập được từ các cảm biến.



Hình 22: Dashboard của webserver và dữ liệu cảm biến

Tiếp theo để gửi dữ liệu lên server Core IoT, Nhóm vào trang Settings để cấu hình kết nối WiFi và máy chủ MQTT.



The screenshot shows the "Device configuration" page in the CORE IOT dashboard. It prompts the user to "Enter your Wi-Fi and Core IoT connection details below." The form includes five input fields: Wi-Fi (containing "ACLAB"), Password (masked with dots), Core IoT Token, MQTT Host (containing "app.coreiot.io"), and MQTT Port (containing "1883"). A "Save Configuration" button is located at the bottom of the form.

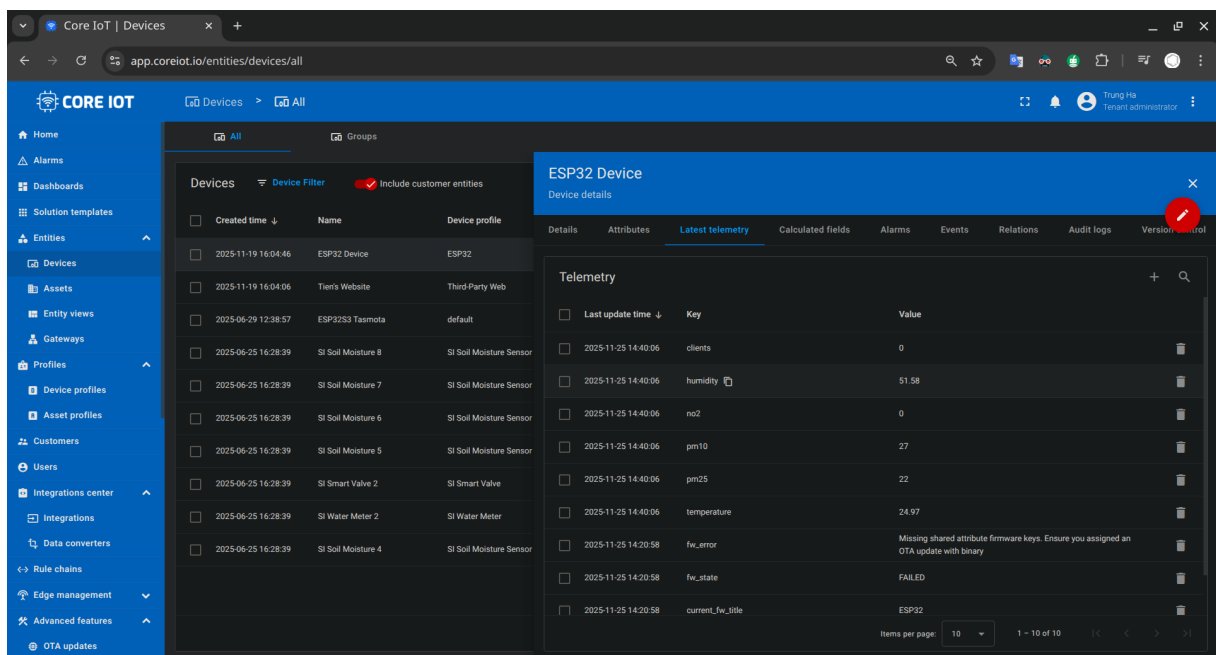
Hình 23: Cấu hình kết nối WiFi và máy chủ MQTT

```
Info: [Webserver] Sent data via WebSocket: {"no2":28.00,"p  
{"page":"setting","value":{"ssid":"OXY 24H 2.4G","password  
Info: [WebSocket] Received configuration from WebSocket:  
SSID: OXY 24H 2.4G  
PASS: oxy24hcoffee  
TOKEN: 2WKWpsCLcXkt7Vffaifn  
SERVER: app.coreiot.io  
PORT: 1883
```

Hình 24: Log khi ESP32 nhận được cấu hình từ người dùng

```
Info: [WiFi] Connecting to WiFi.....  
Info: [WiFi] Connected to WiFi  
WIFI_SSID = OXY 24H 2.4G  
WIFI_PASSWORD = oxy24hcoffee  
Info: [CoreIoT] Connected to Wifi  
Info: [WEBSERVER STOP] Webserver is running: 0  
Info: [MQT] Connecting to: (app.coreiot.io) with token (2WKWpsCLcXkt7Vffaifn)  
ESP320.0.0  
Info: [CoreIoT] Firmware Update Subscription...Done  
Info: [CoreIoT] Subscribing for RPC...Done
```

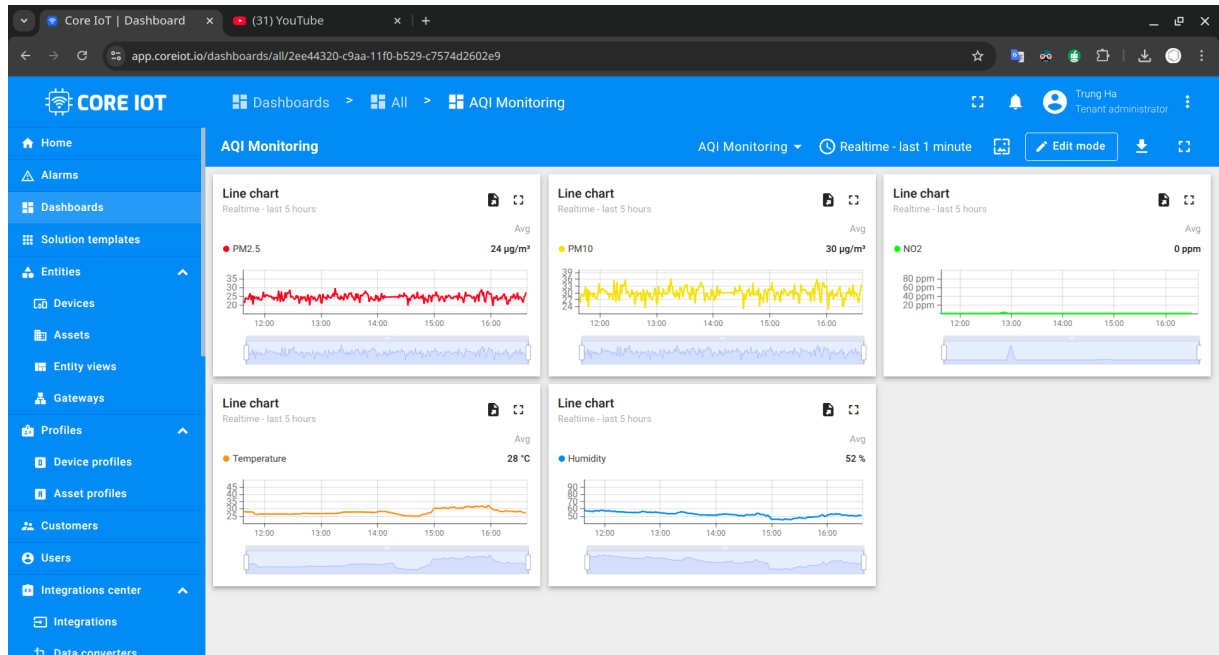
Hình 25: Log khi ESP32 kết nối thành công với WiFi và máy chủ MQTT



The screenshot displays the Core IoT web application interface. On the left is a navigation menu with options like Home, Alarms, Dashboards, Solution templates, Entities, Devices, Assets, Entity views, Gateways, Profiles, Device profiles, Asset profiles, Customers, Users, Integrations center, Integrations, Data converters, Rule chains, Edge management, Advanced features, and OTA updates. The main area shows a list of devices under the 'All' tab. A modal window titled 'ESP32 Device' is open, showing details for a specific device. The 'Latest telemetry' tab is selected, displaying a table of sensor data.

Last update time	Key	Value
2025-11-25 14:40:06	clients	0
2025-11-25 14:40:06	humidity	51.58
2025-11-25 14:40:06	no2	0
2025-11-25 14:40:06	pm10	27
2025-11-25 14:40:06	pm25	22
2025-11-25 14:40:06	temperature	24.97
2025-11-25 14:20:58	fw_error	Missing shared attribute firmware keys. Ensure you assigned an OTA update with binary
2025-11-25 14:20:58	fw_state	FAILED
2025-11-25 14:20:58	current_fw_title	ESP32

Hình 26: Dữ liệu cảm biến được gửi đến và lưu trữ tại server Core IoT



Hình 27: Dữ liệu thu thập được từ cảm biến trong 5 tiếng (11:30 - 16:30)

```
Info: [MQT] Sending telemetry: {"temperature":26.88,"humidity":50.17,"no2":0,"pm25":20,"pm10":24,"clients":0}
Info: [MQT] Sending telemetry: {"temperature":26.88,"humidity":49.66,"no2":0,"pm25":26,"pm10":32,"clients":0}
Info: [CoreIoT] Starting firmware update...
temperature":26.88,"humidity":50.28,"no2":0,"pm25":28,"pm10":35,"clients":0}
Info: [CoreIoT] Progress 6.67%
Info: [CoreIoT] Progress 10.00%
Info: [CoreIoT] Progress 13.33%
Info: [CoreIoT] Progress 16.67%
Info: [CoreIoT] Progress 20.00%
Info: [CoreIoT] Progress 23.33%
Info: [CoreIoT] Progress 26.67%
Info: [CoreIoT] Progress 30.00%
Info: [CoreIoT] Progress 33.33%
Info: [CoreIoT] Progress 36.67%
Info: [CoreIoT] Progress 40.00%
Info: [CoreIoT] Progress 43.33%
Info: [MQT] Sending telemetry: {"temperature":26.89,"humidity":49.99,"no2":0,"pm25":27,"pm10":33,"clients":0}
Info: [CoreIoT] Progress 46.67%
Info: [CoreIoT] Progress 50.00%
Info: [CoreIoT] Progress 53.33%
Info: [CoreIoT] Progress 56.67%
Info: [CoreIoT] Progress 60.00%
Info: [CoreIoT] Progress 63.33%
Info: [CoreIoT] Progress 66.67%
Info: [CoreIoT] Progress 70.00%
Info: [CoreIoT] Progress 73.33%
Info: [CoreIoT] Progress 76.67%
Info: [CoreIoT] Progress 80.00%
Info: [CoreIoT] Progress 83.33%
Info: [MQT] Sending telemetry: {"temperature":26.87,"humidity":49.71,"no2":0,"pm25":25,"pm10":31,"clients":0}
Info: [CoreIoT] Progress 86.67%
Info: [CoreIoT] Progress 90.00%
Info: [CoreIoT] Progress 93.33%
Info: [CoreIoT] Progress 96.67%
Info: [CoreIoT] Progress 100.00%
Info: [CoreIoT] Done, Reboot now
Info: [WiFi] Connecting to WiFi...[228463][E][WiFiSTA.cpp:253] begin(): disconnect failed!
.ESP-ROM:esp32s3-20210327
```

Hình 28: Cập nhật firmware từ xa bằng OTA Update của Core IoT

```
Info: [Tiny ML] Inference result: 0.53
Info: [MQT] Sending telemetry: {"no2":81.
Info: [Tiny ML] Inference result: 0.54
Info: [MQT] Sending telemetry: {"no2":64.
Info: [Tiny ML] Inference result: 0.54
Info: [MQT] Sending telemetry: {"no2":42.
Info: [Tiny ML] Inference result: 0.53
Info: [MQT] Sending telemetry: {"no2":42.
Info: [Tiny ML] Inference result: 0.53
Info: [MQT] Sending telemetry: {"no2":49.
```

Hình 29: Log phát hiện bất thường của Tiny ML

5.2 Đánh giá hiệu năng thực tế

Để kiểm tra mức độ ổn định và khả năng đáp ứng của hệ thống, nhóm đã tiến hành chạy thử nghiệm liên tục trong 5 giờ (11:30–16:30). Các thông số hiệu năng được ghi nhận bao gồm thời gian đọc cảm biến, độ trễ truyền dữ liệu, độ ổn định kết nối và khả năng xử lý yêu cầu điều khiển từ xa. Kết quả thu được như sau:

- **Thời gian đọc cảm biến DHT20:** ~ 40 ms.
- **Thời gian đọc hai cảm biến Modbus RS485 (NO_2 và PM):** ~ 350 ms.
- **Thời gian xử lý và đóng gói dữ liệu trên ESP32:** 5–10 ms.
- **Chu kỳ đọc và xử lý dữ liệu:** ~ 400 ms.
- **Chu kỳ gửi dữ liệu MQTT lên máy chủ CoreIoT:** ổn định ở 2 s.
- **Độ trễ truyền MQTT tới máy chủ:** < 1 s trong điều kiện Internet ổn định.
- **Độ trễ xử lý yêu cầu điều khiển từ dashboard:** < 1 s.
- **Độ ổn định vận hành:** không xuất hiện treo hệ thống, reset ESP32 hoặc mất kết nối MQTT trong suốt 5 giờ thử nghiệm.
- **Giao tiếp RS485:** hoạt động ổn định, không ghi nhận lỗi khung (frame error).

5.3 Đối chiếu với mục tiêu ban đầu

Dựa trên các yêu cầu được đặt ra ở giai đoạn phân tích hệ thống (mục tiêu chức năng và phi chức năng), hiệu năng thực tế cho thấy hệ thống đã đáp ứng tốt các tiêu chí đề ra.

So sánh với yêu cầu chức năng

- **Thu thập dữ liệu đa thông số:** Hệ thống đã đọc ổn định dữ liệu PM2.5, PM10, NO_2 và nhiệt độ/độ ẩm với chu kỳ xử lý ~ 400 ms, đáp ứng tốt yêu cầu thu thập định kỳ.
- **Giao tiếp và truyền thông:** Cơ chế AP/STA hoạt động đúng thiết kế; sau khi kết nối WiFi, dữ liệu được gửi đều đặn lên server với chu kỳ 2 s.
- **Hiển thị và cấu hình:** Website dashboard hiển thị đầy đủ dữ liệu thời gian thực; các tham số cấu hình từ xa được áp dụng ngay lập tức với độ trễ điều khiển < 1 s.
- **Tự động hoá với ngưỡng và AI tại biên:** Hệ thống đã xử lý ngưỡng cục bộ ổn định; độ trễ phản hồi nhỏ hơn 1 s, phù hợp với yêu cầu kích hoạt tức thời.

So sánh với yêu cầu phi chức năng

- **Hiệu suất:** Yêu cầu: độ trễ từ thiết bị đến dashboard không vượt quá giới hạn. Kết quả: độ trễ truyền MQTT luôn < 1 s, hoàn toàn đạt yêu cầu.
- **Độ tin cậy:** Yêu cầu: tự động kết nối lại khi mất mạng, không treo hệ thống. Kết quả: trong suốt thời gian thử nghiệm, thiết bị không treo, không reset và không gián đoạn kết nối.
- **Khả năng mở rộng:** Yêu cầu: kiến trúc modular, dễ thêm cảm biến. Kết quả: hệ thống thực tế đã tích hợp nhiều loại cảm biến (DHT20, NO_2 , PM RS485) mà không cần thay đổi kiến trúc cốt lõi; đạt yêu cầu mở rộng.

6 Kết luận và hướng phát triển

6.1 Tổng kết các kết quả đạt được

Qua quá trình thiết kế, xây dựng và triển khai, nhóm đã hoàn thiện một hệ thống IoT quan trắc môi trường dựa trên vi điều khiển ESP32-S3 hoạt động ổn định và đáp ứng đầy đủ các yêu cầu đề ra. Cụ thể, hệ thống đã đạt được các kết quả sau:

- Thu thập thành công các thông số môi trường quan trọng: nhiệt độ, độ ẩm, NO_2 , $\text{PM}_{2.5}$ và PM_{10} theo chu kỳ định sẵn.
- Thiết kế và triển khai WebServer hoạt động ở chế độ AP, cho phép người dùng cấu hình WiFi, MQTT, ngưỡng cảnh báo và cập nhật firmware một cách dễ dàng.
- Kết nối và truyền dữ liệu ổn định lên nền tảng CoreIoT thông qua giao thức MQTT, hỗ trợ hiển thị dữ liệu thời gian thực trên dashboard.
- Ứng dụng FreeRTOS để tách các chức năng thành các tác vụ độc lập, giúp hệ thống hoạt động mượt, có tính mở rộng và dễ bảo trì.
- Tích hợp TinyML ở mức cơ bản, giúp thiết bị phát hiện bất thường tại biên và gửi cảnh báo lên server.
- Xây dựng website dashboard riêng, kết nối tới CoreIoT để trực quan hóa dữ liệu thu thập được.

Các kết quả trên cho thấy hệ thống đã đáp ứng mục tiêu ban đầu: xây dựng một giải pháp IoT hoàn chỉnh từ phần cứng đến phần mềm và giao diện người dùng.

6.2 Các giới hạn của hệ thống

Mặc dù hệ thống hoạt động tốt ở quy mô thử nghiệm, vẫn tồn tại một số giới hạn và ràng buộc công nghệ:

- **Giới hạn phần cứng:** ESP32-S3 có tài nguyên RAM/Flash hạn chế khi chạy đồng thời nhiều tác vụ phức tạp (FreeRTOS, WebServer, MQTT, TinyML). Các cảm biến công nghiệp RS485 yêu cầu nguồn 12–24V nên hệ thống cần cấp nguồn ngoài.
- **Giới hạn kết nối:** Kết nối WiFi 2.4 GHz dễ bị nhiễu, gây mất ổn định khi có nhiều thiết bị trong cùng mạng. WebServer chạy trên ESP32 xử lý yêu cầu chậm nếu nhiều client truy cập cùng lúc.
- **Giới hạn về độ ổn định và thời gian thực:** Chu kỳ đọc cảm biến RS485 phụ thuộc thời gian phản hồi của từng cảm biến nên không thể giảm quá thấp. TinyML chạy trên thiết bị chỉ phù hợp các mô hình nhỏ; mô hình lớn hơn gây thiếu bộ nhớ.
- **Giới hạn về nền tảng hiển thị:** Website dashboard được xây dựng thủ công nên vẫn còn đơn giản, chưa có khả năng tùy biến giao diện sâu.

Những hạn chế này chủ yếu xuất phát từ giới hạn phần cứng, tính phức tạp của hệ thống nhúng và phạm vi thực hiện trong khuôn khổ bài tập lớn.

6.3 Đề xuất hướng cải tiến

Để mở rộng khả năng ứng dụng thực tế trong tương lai, nhóm đề xuất một số hướng phát triển cho hệ thống:

- **Tăng cường AI/TinyML:** Huấn luyện mô hình phân loại chất lượng không khí trực tiếp trên thiết bị hoặc dự báo dữ liệu (time-series). Sử dụng các mô hình nhẹ (MicroTensor, TFLite Micro) để tối ưu lý luận tại biên.
- **Bổ sung cảnh báo thông minh:** Gửi thông báo qua ứng dụng di động, Telegram, Zalo hoặc email khi thông số vượt ngưỡng. Kết hợp dữ liệu lịch sử để cảnh báo sớm (early warning).

- **Hoàn thiện hệ thống tự động hóa:** Điều khiển tự động quạt, máy lọc, còi báo động dựa trên cảm biến hoặc thuật toán AI. Phát triển cơ chế quy tắc (rule engine) giống ThingsBoard.
- **Mở rộng phần cứng:** Thêm nhiều cảm biến khí khác (CO, O₃, SO₂, CO₂, VOC). Tích hợp pin và sạc năng lượng mặt trời cho các ứng dụng ngoài trời.
- **Nâng cấp giao diện người dùng:** Xây dựng ứng dụng mobile (Flutter/React Native). Cải thiện website dashboard với nhiều biểu đồ và chức năng phân tích dữ liệu nâng cao.
- **Tăng độ ổn định hệ thống:** Thêm watchdog timer, tự động khởi động lại khi lỗi. Tối ưu FreeRTOS, giảm tải WebServer, và áp dụng cơ chế đồng bộ dữ liệu hiệu quả hơn.

Những cải tiến trên sẽ giúp hệ thống hướng tới ứng dụng thực tế với độ ổn định cao hơn, khả năng giám sát chính xác hơn, và trải nghiệm người dùng tốt hơn trong các phiên bản tiếp theo.

7 Mã Nguồn

Mã nguồn của dự án được lưu tại [ESP32-IoT-Monitoring-System-using-PlatformIo](#) và [Website-IoT-Monitoring-System-using-NodeJS](#)

Tài liệu tham khảo

- [1] O. Education, *Giới thiệu về yolo uno*, https://docs.ohstem.vn/en/latest/yolo_uno/yolo_uno_khoi_lenh/gioi_thieu_yolo_uno.html, 2022.
- [2] E. Systems, *Esp32-s3 series – datasheet*, https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf, 2022.
- [3] ASAIR, *Dht20 – humidity and temperature module datasheet*, <https://aqicn.org/air/sensor/spec/asair-dht20.pdf>, 2021.
- [4] O. Education, *Cảm biến nhiệt độ và độ ẩm dht20*, <https://docs.ohstem.vn/en/latest/module/cam-bien/dht20.html>, 2023.
- [5] E. Systems, *Esp-arduino (arduino core for esp32)*, <https://www.espressif.com/en/sdks/esp-arduino>, 2025.
- [6] FreeRTOS, *What is freertos?* <https://www.freertos.org/Why-FreeRTOS/What-is-FreeRTOS>, Truy cập ngày 21/11/2025, 2025.
- [7] E. Systems, *Esp32-wroom-32 datasheet*, 2021. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf (visited on 11/16/2025).
- [8] PlatformIO, *Platformio official documentation*, 2025. [Online]. Available: <https://docs.platformio.org/> (visited on 11/16/2025).
- [9] Arduino, *Esp32 wi-fi and webserver tutorials*, 2025. [Online]. Available: <https://randomnerdtutorials.com/projects-esp32/> (visited on 11/16/2025).
- [10] TensorFlow, *Tensorflow lite for microcontrollers*, 2025. [Online]. Available: <https://www.tensorflow.org/lite/microcontrollers> (visited on 11/16/2025).
- [11] D. Minoli, *Building the Internet of Things with ESP32*. Wiley, 2020.
- [12] M. Richardson and S. Wallace, *Getting Started with Raspberry Pi and ESP32 for IoT Projects*. O'Reilly, 2019.
- [13] P. Gupta *et al.*, “Anomaly detection in iot sensor data using tinymml,” *IEEE Access*, 2021.
- [14] GitHub, *Esp32 example repositories*, 2025. [Online]. Available: <https://github.com/espressif/esp-idf> (visited on 11/16/2025).
- [15] C. IoT, *Core iot là gì?* <https://coreiot.io/docs/core-iot-la-gi/>, 2025.
- [16] T. C. Edition, *Mqtt device api reference*, <https://thingsboard.io/docs/reference/mqtt-api/>, 2025.